- **基本操作**
  - SQL (Structured Query Language)语言
    - 组成
      - 数据定义语言, DDL (Data Definition Language, DDL)
      - 数据操纵语言 DML (Data Manipulation Lauaguage, DML)
      - 数据控制语言DCL (Data Control Lauguage, DCL)
    - 基本功能
      - 数据定义
        - CREATE
        - DROP
        - ALTER
      - 数据查询
        - SELECT
      - 数据操纵
        - INSERT
        - UPDATE
        - DELETE
      - 数据控制
        - GRANT
        - REVOKE
  - MySQL基本操作
    - 创建数据库
      - 创建数据库命令
      - 设置数据库字符集的命令
      - 查看数据库文件存放路径的命令
    - 查看数据库
    - 选择当前数据库
    - 删除数据库
    - 创建数据库表
      - 常用关键字
        - AUTO_INCREMENT
        - DEFAULT
        - NOT NULL
        - UNSIGNED
        - PRIMARY KEY
        - UNIQUE
        - FOREIGN KEY
    - 查看数据库表结构
    - 修改数据库表结构
    - 删除数据库表
- **管理表记录**
    - MySQL基本数据类型
      - 整数类型
      - 小数类型
      - 字符串类型
      - 日期时间类型
      - 复合类型
    - MySQL运算符
      - 算术运算符
      - 比较运算符
      - 逻辑运算符

- 位运算符
- 运算符的优先级
- 字符集设置
- 增添表记录
  - INSERT
  - REPLACE
  - UPDATE
- 删除表记录
  - DELETE
  - TRUNCATE

检索表记录

- 单表查询
  - 基本查询语句
  - 条件查询语句
    - 使用关系表达式查询
    - 使用逻辑表达式查询
    - 设置取值范围的查询
    - 空值查询
    - 模糊查询
  - 分组查询语句
    - 聚集/合函数
    - Group by 子句
    - having子句
- 多表查询
  - 表连接
    - 内连接（**INNER JOIN** ）
    - 外连接（**OUTER JOIN** ）
      - 左外连接（LEFT OUTER JOIN ）
      - 右外连接（RIGHT OUTER JOIN ）
    - 自连接（非自然连接）
    - 交叉连接(CROSS JOIN)
    - 子查询
      - 返回单行的子查询
      - 返回多行的子查询
    - 子查询与数据更新
      - INSERT
      - UPDATE
      - DELETE
    - 联合查询（集合查询一种）
- 高级查询（复杂子查询）
  - 包含关系查询
  - 报表汇总查询

视图（view）

- 定义
- 优点
  - 保护数据安全
  - 简化操作
  - 使分散数据集中
  - 提高数据的逻辑独立性
- 创建视图
  - 创建视图语法
    - or replace

algorithm 子句
merge
temptable
undefined
view_name
column_list
select_statement
with check option
定义单源表视图
定义多源表视图
在已有视图上创建新视图
创建带表达式的视图
含分组统计信息的视图
创建视图注意事项
权限
子查询
系统或者用户变量
预处理语句参数
在存储子程序内
引用的表/视图的存在问题
temporary 表
在视图定义中命名的表必须已经存在
不能将触发程序与视图关联在一起
order by相关问题
查看视图
describe 语句
show table status 语句
show create view 语句
查看某数据库下的视图
管理视图
修改视图
删除视图
使用视图
使用视图查询数据
使用视图更新数据
视图缺点
性能
表依赖关系
MySQL 视图注意点
MySQL 5.x 版本之后支持数据库视图
不能在视图上创建索引
一个简单的视图可以更新表中数据。基于具有连接，子查询等的复杂 SELECT语句创建的视图无法更新
MySQL 不像 Oracle， PostgreSQL 等其他数据库系统那样支持物理视图， MySQL 是不支持物理视图的
存储过程（PROCEDURE）
MySQL 存储过程的优点
提高性能
减少流量
可重用和透明
安全
MySQL 存储过程的缺点
增加内存使用量
很难调试存储过程

- 开发、维护、移植困难
- 创建与修改存储过程
- 存储过程基本语法
  - 存储过程说明
    - 权限问题
    - 关联问题
    - 参数说明
    - Characteristic 特性说明
    - 过程体说明
    - BEGIN ... END 复合语句块
- 修改和删除存储过程
  - 修改语法
  - 删除语法
- 查看存储过程
- 使用变量
  - 1 DECLARE局部变量
  - 2 SET语句为变量复制
  - 3 SELECT ... INTO 语句
- 流程控制语句
  - IF语句
  - CASE 语句
  - WHILE循环
  - REPEAT 循环
  - LOOP,LEAVE 和 ITERATE 语句
- 定义条件和处理
  - 1 DECLARE 条件的定义
  - 2 条件的处理
  - 其他条件形式
- 存储过程示例

函数（FUNCTION）

- 函数示例
- 修改和删除函数
  - 修改语法
  - 删除语法

游标

- 定义
  - MySQL 游标为只读，不可滚动和敏感
- 使用游标
  - 基本步骤
    - 1.声明游标
    - 2.打开游标
    - 3.使用游标
    - 4.关闭游标
  - 说明
- 游标示例

触发器

- 定义
- 触发器的使用
  - 安全性
  - 审计
  - 实现复杂的数据完整性规则
  - 实现复杂的非标准的数据库相关完整性规则

- 创建并使用触发器
  - 创建触发器
  - 使用触发器
  - 查看触发器
    - show triggers
    - information_schema.triggers
    - show create trigger
    - 记事本
  - 删除触发器
- 触发器的应用
  - 触发器使用注意事项
  - 使用触发器的例子
    - 维护冗余数据
    - 使用触发器模拟外键级联选项
- 触发器示例

JDBC连接数据库

- JDBC 概述
  - JDBC 的基本功能
  - 常见的 JDBC 驱动程序可分为4种类型
    - Type1:JDBC- ODBC Bridge driver
    - Type2:native- API,partly Java driver
    - Type3:JDBC-Net pure Java driver
    - Type4:native protocol, pure Java driver
- JDBC 连接过程
  - MySQL Connector/J
  - JDBC 连接 MySQL 数据库步骤:
    - 加载 JDBC 驱动程序
    - 创建数据库连接
    - 创建 Statement 对象
      - 创建 Statement 对象
      - 创建 PreparedStatement 对象
    - 执行 SQL 语句
    - 处理执行 SQL 语句的返回结果
    - 关闭连接
- JDBC 数据库操作
  - 增加数据
  - 修改数据
  - 删除数据
  - 查询数据
  - 批处理
    - 批量执行静态 SQL
    - 批量执行动态的 SQL
    - 批量执行混合模式的 SQL
- 数据源
- JDBC示例

索引

  - 示例

事务管理

      - 银行例子:
      - 事务的提交和回滚
      - 事务的四大特性和隔离级别
    - 事务的隔离级别

脏读
不可重复读
幻读
code

基本操作

SQL (Structured Query Language)语言

组成

数据定义语言, DDL (Data Definition Language, DDL)

数据操纵语言 DML (Data Manipulation Lauaguage, DML)

数据控制语言DCL (Data Control Lauguage, DCL)

基本功能

数据定义

CREATE

DROP

ALTER

数据查询

SELECT

数据操纵

INSERT

UPDATE

DELETE

数据控制

GRANT

REVOKE

MySQL基本操作

创建数据库

创建数据库命令

```
CREATE DATABASE newdatabase;
```

设置数据库字符集的命令

```
ALTER DATABASE newdatabase CHARACTER SET utf8;
```

查看数据库文件存放路径的命令

```
SHOW VARIABLES LIKE 'datadir'
```

查看数据库

（查看当前MySQL服务器上的**数据库列表**的命令）

```
SHOW databases
```

选择当前数据库

```
USE A;
```

删除数据库

```
DROP DATABASE A;
```

创建数据库表

```
CREATE TABLE [IF NOT EXISTS] 表名
( 字段名1 数据类型 [ [ 约束] ]]
, [ , 字段名2 数据类型 [ [ 约束] ]]
……
, [ , 字段名n 数据类型 [ [ 约束] ]]
[ [ 其他约束条件] ]
[ )[
```

eg

```
use bookstore;
create table users
(
uid int not null primary key auto_increment,
name varchar(20) not null unique,
pwd varchar(20) not null,
sex char(2)
);
```

常用关键字

AUTO_INCREMENT

用于设置整数类型字段的自动增量属性。 AUTO_INCREMENT 字段 必须被索引 ， 而且必须为NOT NULL 。 每个表 最多只能有一个字段 具有AUTO_INCREMENT

eg

```
uid int not null primary key auto_increment
```

DEFAULT

表中添加新行时给表中某一字段指定的默认值。使用 DEFAULT 定义，一是可以避免 NOT NULL 值的数据错误；二是可以加快用户的输入速度

eg

```
pwd varchar(20) not null default female
```

NOT NULL

指定 NOT NULL 属性的字段，不能有 NULL

```
name varchar(20) not null unique
```

UNSIGNED

表示该值不能为负数

PRIMARY KEY

主键

```
uid int not null primary key auto_increment
```

UNIQUE

唯一索引

```
name varchar(20) not null unique
```

FOREIGN KEY

```
create table emp
(
foreign key(uid) references user(uid),
##emp.uid参照user.uid
...
);
```

查看数据库表结构

```
describe users;
```

显示表

修改数据库表结构

```
ALTER TABLE 表名 ACTION [,ACTION]…
```

✓ 添加、修改、删除表属性字段
✓ 添加、修改、删除表约束条件
✓ 删除索引
✓ 修改表名
✓ 修改表的存储引擎

删除数据库表

```
DROP TABLE [IF EXISTS] 表名 1 [, 表名,...];
```

eg

```
DROP TABLE users;
```

管理表记录

MySQL基本数据类型

整数类型

类型	字节数	有符号数范围	无符号数范围
TINYINT	1 字节	-128～+127	0～255
SMALLINT	2 字节	-32768～+32767	0～65535
MEDIUMINT	3 字节	-8388608～+8388607	0～16777215
INT（INTEGER）	4 字节	-2147483648～+2147483647	0～4294967295
BIGINT	8 字节	-9223372036854775808～+9223372036854775807	0～18446744073709551615

默认情况下，整数类型既可以表示正整数，也可以表示负整数

只希望表示正整数

```
age tinyint unsigned
```

指定其显示宽度

```
int(8)
##数值宽度小于8 8 位时在数字前面填满宽度
```

zerofill：在数字位数不够时需要用“ 0” 填充

插入的整数位数大于指定的显示宽度时，将按照整数的**实际值**进行存储

AUTO_INCREMENT ： 在需要产生唯一标识符或顺序值时，可以利用此属性，该属性只适用于整数类型。一个表中最多只能有一个 AUTO_INCREMENT 字段，该字段应该为 NOT NULL ， 并且定义为 PRIMARY KEY 或 UNIQUE 。AUTO_INCREMENT 字段值从1开始，每行记录其值增加1 。当插入 NULL 值到一个 AUTO_INCREMENT 字段时，插入的值为该字段中当前最大值加1。

小数类型

浮点数和定点数

浮点数包括单精度浮点数 FLOAT 类型和双精度浮点数DOUBLE 类型，定点数为 DECIMAL 类型

定点数在 MySQL 内部以**字符串**形式存放，比浮点数更精确，适合用来表示货币等**精度高**的数据

浮点数和定点数都可以在类型后面加上 （ M ， D ） 来表示，M 表示该数值**一共**可显示 M 位数字，D 表示该数值小数点后的位数

在类型后面指定 （ M ， D ） 时，小数点后面的数值需要按照D来进行四舍五入

当不指定 （ M ， D D ） 时，浮点数将按照实际值来存储，而DECIMAL 默认的整数位为 10 ， 小数位为0

字符串类型

类型	存储长度范围
CHAR(n)	0～255
VARCHAR(n)	0～255（5.0.3 版本以前）或 0～65535（5.0.3 版本以后）
TINYTEXT	0～255
TEXT	0～65535
MEDIUMTEXT	0～16777215
LONGTEXT	0～4294967295

CHAR 与 VARCHAR 都是用来保存 MySQL 中较短的字符串，二者的主要区别在于存储方式不同。CHAR(n) 为定长字符串类型，n n 的取值为0 0 ～ 255 ； VARCHAR(n) 为变长字符串类型，n n 的取值为0 0 ～ 255 （ 5.0.3 版本以前）或0 0 ～ 65535 （ 5.0.3 版本以后）。 CHAR(n) 类型的数据在存储时会删除尾部空格，而 VARCHAR(n) 在存储数据时则会保留尾部空格。

日期时间类型

类型	最小值	最大值
DATE	1000-01-01	9999-12-31
TIME	-838:59:59	838:59:59
DATETIME	1000-01-01 00:00:00	9999-12-31 23:59:59
TIMESTAMP	1970-01-01 08:00:01	2037 年的某个时刻
YEAR	1901	2155

在 YEAR 类型中，年份值可以为2位或4位，默认为4位。在4位格式中，允许值的范围为 1901 ～ 2155 。在2位格式中，取值为 70 ～ 99 时，表示从 1970 年～ 1999 年；取值为 01 ～ 69 时，表示从 2001 年～ 2069 年

DATETIME 与 TIMESTAMP 都包括日期和时间两部分，但TIMESTAMP 类型与**时区**相关，而 DATETIME则与时区无关

如果在一个表中定义了两个类型为 TIMESTAMP 的字段，则表中第一个类型为 TIMESTAMP 的字段其默认值为
CURRENT_TIMESTAMP ，第二个 TIMESTAMP 字段的默认值为 0000- - 00- - 00 00:00:00 。

复合类型

MySQL 中的复合数据类型包括： **ENUM** 枚举类型和 **SET** 集合类型。ENUM 类型只允许从集合中取得某一个值， SET 类型允许从集合中取得多个值。 ENUM 类型的数据最多可以包含 65535 个元素， SET 类型的数据最多可以包含 64 个元素。

MySQL运算符

算术运算符

运算符	说明
+	加法运算
-	减法运算
*	乘法运算
/	除法运算
%	取余运算

比较运算符

运算符	说明
=	等于
!=或<>	不等于
<=>	NULL 安全的等于
<	小于
<=	小于等于
>	大于
>=	大于等于
IS NULL	为 NULL
IS NOT NULL	不为 NULL
BETWEEN AND	在指定范围内
IN	在指定集合内
LIKE	通配符匹配
REGEXP	正则表达式匹配

```
varchar_field like '%Li'
##字符串是否以“ Li ”结尾
```

```
varchar_field regexp '^Mr'
##字符串是否以“ Mr ”开头
```

```
varchar_field regexp 'Li$'
##字符串是否以“ Li ”结尾
```

逻辑运算符

逻辑非（NOT 或！）、逻辑与（AND 或 &&）、逻辑或（OR 或 ||）和逻辑异或（XOR）

逻辑异或（XOR）：**当任意一个操作数为 NULL 时，逻辑异或的返回值为 NULL 。**对于非 NULL 操作数，如果两个操作数的逻辑真假值相异，则返回结果为1；否则返回值为0。

位运算符

按位与（&）、按位或（|）、按位取反（~）、按位异或（^）

左移（<<）和右移（>>）

运算符的优先级

优先级	运算符
1	!
2	~，-
3	^
4	*，/，DIV，%，MOD
5	+，-
6	>>，<<
7	&
8	|
9	=，<=>，<，<=，>，>=，!= ，<> ，IS，IN，LIKE，REGEXP
10	BETWEEN AND，CASE，WHEN，THEN，ELSE
11	NOT
12	&&，AND
13	||，OR，XOR
14	:=

字符集设置

字符排序规则（collation）是指在同一字符集内字符之间的比较规则，一个字符集可以包含多种字符排序规则，每个字符集会有一个默认的字符排序规则

MySQL 中字符排序规则命名方法为：以字符排序规则对应的字符集开头，中间是国家名（或 general），以 ci 、cs 或 bin 结尾。以**ci 结尾的字符排序规则表示大小写不敏感**，以**cs 结尾的字符排序规则表示大小写敏感，以 bin 结尾的字符排序规则表示按二进制编码值进行比较。**

在字符集utf-8下，要满足大小写敏感，应采用排序规则____:

A、 gbk_chinese_cs

B、 utf8_general_ci

C、 utf8_general_cs

D、 utf8_general_bin

正确答案： C　　我的答案：C

增添表记录

INSERT

使用 INSERT 语句可以将一条或多条记录插入表中，也可以将另一个表中的结果集插入到当前表中。

```
INSERT INTO employee
VALUES(19, null, '四', '李', '2022-9-22', '出纳员', 1, 2, 4);
```

```
insert into course values(NULL,'C 程序设计 ',default,'0412893402');
insert into course( course_name,teacher_id ) values('Java 程序设计','0412893401');
insert into course values( NULL,'MySQL 数据库',60,'0412893403');
```

```
INSERT INTO
employee(EMP_ID,END_DATE,FIRST_NAME,LAST_NAME,START_DATE,TITLE,ASSIGNED_BRANCH_ID,SU
PERIOR_EMP_ID,DEPT_ID)
SELECT 20,NULL,FIRST_NAME,LAST_NAME,'2022-9-26','出纳员', 1, 2, 4 FROM individual
WHERE CUST_ID=1;
```

REPLACE

使用 REPLACE 语句也可以将一条或多条记录插入表中，或者将一个表中的结果集插入到目标表中。

使用 REPLACE 语句添加记录时，如果新记录的主键值或者唯一性约束的字段值与已有记录相同时，则已有记录被删除后再添加新记录。

```
replace into teacher values('0412893404',' 唐明明','13401234567');
```

UPDATE

当记录插入后，可以使用 UPDATE 语句对表中的记录进行修改

```
UPDATE employee
SET TITLE = '出纳主任'
WHERE EMP_ID = 19;
```

删除表记录

DELETE

```
DELETE from table_name
[WHERE condition]
```

在上面 DELETE 语句中，如果没有指定 WHERE 子句，则表中所有记录都被删除，但**表结构仍然存在**

使用 DELETE删除数据时，会有一个返回值，其含义是**被删除的记录数目**

eg

```
DELETE FROM acc_transaction
Where TXN_DATE < '2012-01-01';
```

TRUNCATE

```
TRUNCATE [table] table_name
```

TRUNCATE table 语句清空表记录后会**重新设置自增型字段的计数起始值**，但 DELETE 语句则不会。

检索表记录

单表查询

基本查询语句

SELECT...FROM查询语句

```
SELECT [ALL | DISTINCT] < 目标列表达式> [ , < 目标列表达式>]
FROM < 表名或视图名>[ , < 表名或视图名>]
[WHERE < 条件表达式>]
[GROUP BY < 列名1> [HAVING < 条件表达式>]]
[ORDER BY < 列名2> [ASC | DESC]]
[LIMIT [start,] count];
```

ALL 关键字表示将会显示所有检索的数据行，包括重复的数据行（默认）

DISTINCT 关键字表示仅显示不重复的数据行，对于重复的数据行，则只显示一次

LIMIT 10,20 表示从结果集的第**11** 行记录开始返回20行记录

条件查询语句

where 子句

使用关系表达式查询

关系表达式是指在表达式中含有关系运算符

常见的关系运算符有：= （等于）、> （大于）、< （小于）、>= （大于等于）、<= （小于等于）、!= 或 **<>** （不等于）

```
SELECT B_ID,B_Name, B_SalePrice FROM BookInfo
WHERE B_SalePrice>40;
```

使用逻辑表达式查询

常用的逻辑运算符有AND 、OR 和NOT

当一个WHERE 子句同时包括若干个逻辑运算符时，其优先级从高到低依次为NOT 、AND 、OR

如果想改变优先级，可以使用括号

```
SELECT B_ID,B_Name, B_SalePrice
FROM BookInfo
WHERE B_SalePrice>=20 AND B_SalePrice<=40;
```

设置取值范围的查询

谓词BETWEEN ...AND 和NOT BETWEEN ...AND 可以用来设置查询条件。（建议使用>=, <= ）

```
SELECT B_ID,B_Name, B_SalePrice
FROM BookInfo
WHERE B_SalePrice BETWEEN 20 AND 40;
```

空值查询

NULL 是特殊的值，代表“无值”，与0 、空字符串或仅仅包含空格都不相同

在涉及空值的查询中，可以使用IS NULL 或者IS NOT NULL

```
SELECT U_ID,U_Name
FROM Users
WHERE U_Phone IS NULL;
```

模糊查询

要实现模糊查询，必须使用**通配符**，利用通配符可以创建和特定字符串进行比较的搜索模式

%:代表任意多个字符

_:代表任意的一个字符

如果查询条件中使用了通配符，则操作符必须使用LIKE 关键字，用于搜索与特定字符串相匹配的字符数据，其基本的语法形式为

```
[NOT] LIKE < 匹配字符串>
```

```
SELECT B_Name,B_Publisher,B_SalePrice
FROM BookInfo
WHERE B_Name LIKE '%MySQL%';
```

```
SELECT B_Name,B_Author,B_Publisher
FROM BookInfo
WHERE B_Author LIKE '_ 国%';
```

如果要查询的字符串本身就含有通配符，此时就需要用**ESCAPE** 关键字，对通配符进行转义

```
SELECT * FROM Users
WHERE U_Name LIKE '%/_%' ESCAPE '/';
##查询会员名中含有“_” 的会员信息
```

分组查询语句

聚集/合函数

聚合函数	说明
SUM（）	返回某列所有值的总和
AVG（）	返回某列的平均值
MAX（）	返回某列的最大值
MIN（）	返回某列的最小值
COUNT（）	返回某列的行数

除了Count(*) 之外聚合函数忽略列值为NULL 的行

```
SELECT COUNT(*) FROM Users;
##统计Users 表中会员的数量
SELECT COUNT(U_Phone) FROM Users;
##统计填写电话号码的会员个数
```

Group by 子句

```
SELECT B_Publisher ,COUNT(*) AS 总数
FROM BookInfo
GROUP BY B_Publisher;
## 检索BookInfo 表，查询每个出版社出版的图书的数量
```

```
SELECT B_Publisher,MAX(B_MarketPrice) AS 最高价格,MIN(B_MarketPrice) AS 最低价格
FROM BookInfo
GROUP BY B_Publisher;
##检索BookInfo 表，查询每个出版社图书的最高价格和最低价格
```

having子句

如果分组**以后**要求按一定条件对这些组进行筛选，则需要使用HAVING 子句指定筛选条件

```
SELECT B_Publisher ,COUNT(*) AS 总数
FROM BookInfo
WHERE B_MarketPrice>=50
GROUP BY B_Publisher
HAVING COUNT(*)>=2;
##检索BookInfo 表，查询出版了2本及2本以上并且价格大于等于50元的图书信息。
```

Having 子句 vs. where 子句

HAVING 子句和WHERE 子句都是设置查询条件，但两个子句的作用对象不同,WHERE 子句作用的对象是基**本表或视图**，从中选出满足条件的记录；而HAVING 子句的作用对象是**分组**，从中选出满足条件的分组，WHERE 在数据分组之前进行过滤，而HAVING 在数据分组之后进行过滤

多表查询

表连接

```
SELECT <查询列表>
FROM <表名1> [连接类型] JOIN <表名2> ON <连接条件>
WHERE <查询条件>
```

连接类型 有3 种：内连接（**INNER JOIN** ）、外连接（**OUTER JOIN** ）和交叉连接（**CROSS JOIN** ）

用来连接两个表的条件称为**连接条件**，通常是通过匹配多个表中的**公共字段**来实现的

内连接（INNER JOIN ）

内连接是最常用的连接类型，也是默认的连接类型，在FROM 子句中使用INNER JOIN （INNER 关键字可以省略）来实现内连接

```
SELECT B_Name, BookInfo.BT_ID, BT_Name
FROM BookInfo INNER JOIN BookType
ON BookInfo.BT_ID= BookType.BT_ID
ORDER BY BT_ID;
##检索BookInfo和BookType表，查询每本图书所属的图书类别
```

```
SELECT U.U_Name,O.O_ID, O.O_Time, O.O_TotalPrice
FROM Users U INNER JOIN Orders O
ON U.U_ID = O.U_ID
WHERE O_TotalPrice>100;
##检索Users 和Orders 表，查询订单总价超过100
```

外连接（OUTER JOIN ）

使用外连接时，以主表中每行的数据去匹配从表中的数据行，如果符合连接条件则返回到结果集中；如果没有找到匹配行，则主表的行仍然保留，并且返回到结果集中，相应的从表中的数据行被填上NULL 值后也返回到结果集中

外连接有3 种类型，分别是左外连接（LEFT OUTER JOIN ）、右外连接（RIGHT OUTER JOIN ）和全外连接（FULL OUTER JOIN ）

MySQL 暂不支持全外连接

左外连接（LEFT OUTER JOIN ）

左外连接的结果集中包含左表（JOIN 关键字左边的表）中所有的记录，如果右表中没有满足连接条件的记录，则结果集中右表中的相应行数据填充为NULL

```
SELECT U.U_ID,U.U_Name, O.O_Time,O.O_TotalPrice
FROM Users U LEFT OUTER JOIN Orders O
ON U.U_ID = O.U_ID
ORDER BY U_ID;
```

右外连接（RIGHT OUTER JOIN ）

右外连接的结果集中包含满足连接条件的所有数据和右表（JOIN 关键字右边的表）中不满足条件的数据，左表中的相应行数据为NULL

```
SELECT OD.OD_ID,OD_Number,BI.B_ID,BI.B_Name
FROM OrderDetails OD RIGHT OUTER JOIN BookInfo BI
ON OD.B_ID = BI.B_ID;
```

自连接（非自然连接）

在同一个表中进行的连接被称为自连接，可以看作是这张表的两个副本之间进行的连接，必须为该表指定两个别名

```
SELECT B2.B_ID,B2.B_Name,B2.B_SalePrice
FROM BookInfo B1 INNER JOIN BookInfo B2
ON B1.B_Name='C# 基础与案例开发详解' AND
B1.B_SalePrice<B2.B_SalePrice
ORDER BY B2.B_SalePrice DESC;
##要查询BookInfo 表中高于“C# 基础与案例开发详解”会员价格的图书号、图书名称和图书会员价格，查询后的结果集要求按会员价格降序排列
```

交叉连接(CROSS JOIN)

如果不带WHERE 子句时，则返回的结果是被连接的两个表的笛卡尔积

如果交叉连接带有WHERE 子句时，则返回结果为连接两个表的笛卡尔积减去WHERE 子句所限定而省略的行数

```
SELECT O.O_ID,OD.OD_ID
FROM Orders O CROSS JOIN OrderDetails OD;
```

子查询

子查询是指在一个外层查询中包含另一个内层查询，即在一个SELECT 语句中的WHERE 子句 中，包含有另一个SELECT语句

外层的SELECT 语句称为**主查询**， WHERE 子句中包含的SELECT 语句称为**子查询**

一般将子查询的查询结果作为主查询的**查询条件**

返回单行的子查询

返回单行的子查询是指子查询的查询结果只返回一个值或一行值，并将这个返回值作为父查询的条件，在父查询中进一步查询。在WHERE 子句中可以使用比较运算符来连接子查询

```
SELECT OD_ID,OD_Number,OD_Price
FROM OrderDetails WHERE B_ID=
(SELECT B_ID FROM BookInfo WHERE B_Name='ASP.NET 网站开发项目化教程');
```

返回多行的子查询

返回多行的子查询就是子查询的查询结果中包含多行数据； 返回多行的子查询经常与IN 、EXISTS 、ALL 、ANY 和SOME 关键字

IN关键字

```
WHERE <表达式> [NOT] IN (<子查询>)
```

```
SELECT U_ID,U_Name,U_Phone
FROM Users
WHERE U_ID IN
(SELECT U_ID FROM Orders WHERE O_TotalPrice<50);
##查询订单总价小于50 元的会员信息
```

EXISTS关键字

使用EXISTS 关键字的子查询并不返回任何数据，只返回**逻辑真值**和**逻辑假值**

在使用EXISTS 时， **子查询通常将“*” 作为输出列表**

```
WHERE [NOT] EXISTS (<子查询>)
```

```
SELECT U.U_ID,U.U_Name,U.U_Sex
FROM Users U
WHERE EXISTS
(SELECT * FROM Orders O WHERE O.U_ID=U.U_ID);
##查询订购了图书的会员信息
```

ALL 、ANY 和SOME 关键字

ANY 关键字表示**任何一个**（其中之一），只要与子查询中一个值相符合即可

ALL 关键字表示所有（全部），要求与子查询中的**所有值相符合**；**SOME 同ANY 是同义词**

```
SELECT O_ID,O_UserName,O_Address,O_PostCode
FROM Orders WHERE O_ID = ANY
(SELECT O_ID FROM OrderDetails WHERE B_ID>3);
##查询订购了图书编号大于3 的订单编号及收货人的姓名、地址、邮编
```

子查询与数据更新

INSERT

```
INSERT INTO <表名> [<列名>] <子查询>
```

要插入数据的**表**必须已经**存在**

要插入数据的表结构必须和子查询语句的结果集结构相**兼容**

UPDATE

```
UPDATE BookInfo SET B_SalePrice=B_MarketPrice*0.7
WHERE 'MySQL'=
(SELECT BT_Name FROM BookType WHERE
BookInfo.BT_ID=BookType.BT_ID);
## 将BookInfo 表中“MySQL ”类别图书的会员价格修改为市场价格的70% 。
```

DELETE

```
DELETE FROM BookInfo
WHERE 'Java'=
(SELECT BT_Name FROM BookType WHERE
BookInfo.BT_ID=BookType.BT_ID);
##删除BookInfo 表中“Java ”类别图书基本信息
```

联合查询（集合查询一种）

联合查询是指合并两个或多个查询语句的结果集 。

```
SELECT 语句1 UNION [ALL] SELECT 语句2
```

ALL 选项表示保留结果集中的重复记录， **默认时系统自动删除重复记录**

```
SELECT U_Name,U_Phone FROM Users
UNION ALL
SELECT O_UserName,O_Phone FROM Orders;
```

高级查询（复杂子查询）

独立子查询 vs. 相关子查询

独立子查询:子查询的执行与外层无关。可以将子查询的结果带入上层查询的条件，再进行上层查询

相关子查询（理解为双重循环):子查询的执行与外层相关。外层查询的每一次迭代，内层的子查询要根据外层查询当前数据行重新执行一遍（**子查询中需要用到外层查询中的值**）

包含关系查询

```
select Sname from Student where not exists
(select * from Course where not exists
(select * from SC
where Sno=Student.Sno AND Cno=Course.Cno ));
##查询选修了全部课程的学生姓名
#1.选出每一个课没参加的学生
#2.在所有学生中排除这些学生
```

双重否定解析：SQL 语言中没有全称量词 (For all)，可把带有全称量词的谓词转换为等价的双嵌套not exists

报表汇总查询

GROUP BY 子句中的ROLLUP 与CUBE

```
GROUP BY 分组列表达式1[,... 分组列表达式n]
[WITH {CUBE|ROLLUP}]
```

如果是GROUP BY **ROLLUP(A,B,C)**，首先会对(A,B,C) 进行GROUP BY，然后对(A,B) 进行GROUP BY，然后是(A)进行GROUP BY，最后对全表进行GROUP BY 操作

ORDER BY 不能在rollup

如果分组中的列包含NULL 值，rollup 的结果可能不正确，其原因在于rollup 进行分组统计时，null 具有特殊意义。因此在进行rollup 时可以先将null 转换成一个不可能存在的值，或者没有特别含义的值，比如：IFNULL(xxx,0)

如果是GROUP BY **CUBE(A,B,C)**， 首先会对(A,B,C) 进行GROUP BY， 然后是(A,B)，(A,C)，(B,C)，(A)，(B)，(C)， 最后对全表进行GROUP BY操作

视图（view）

定义

视图是用于创建**动态表**的静态定义，视图中的数据是根据预定义的选择条件从一个或多个行集中生成的。用视图可以定义一个或多个表的行列组合。为了得到所需要的行列组合的视图，可以使用 select 语句来指定视图中包含的行和列

视图是一个**虚拟**的表，其结构和数据是建立在对表的查询基础上的，也可以说视图的内容由查询定义，而视图中的数据并不像表、索引那样需要占用存储空间，视图中**保存的仅仅是一条 select 语句**，其数据来自于视图所引用的数据库表或者其他视图，对视图的操作与对表的操作一样，可以对其进行查询、修改、删除。

优点

保护数据安全

视图可以作为一种**安全机制**，同一个数据库可以创建不同的视图，为不同的用户分配不同的视图，通过视图用户只能查询或修改他们所能看到的数据，其他数据库或者表既不可见也不可以访问，增强数据的安全访问控制。

简化操作

视图向用户**隐藏了表与表之间的复杂的连接操作**，大大**简化**了用户对数据的操作。

使分散数据集中

当用户所需的数据分散在数据库多个表中时，通过定义视图可以将这些数据集中在一起，以方便用户对分散数据的**集中查询与处理**。

提高数据的逻辑独立性

有了视图之后，**应用程序可以建立在视图**之上，从而使应用程序和数据库表结构在一定程度上实现逻辑分离。

创建视图

创建视图需要具有针对视图的 create view 权限，以及针对由 select 语句选择的每一列上的某些权限。对于在 select 语句中其他地方使用的列，必须具有elect 权限。如果还有 **or replace** 子句，必须在视图上具有 drop 权限。

创建视图语法

使用 create view 语句来创建视图语法格式为

```
create
[or replace]
[algorithm={undefined |merge | e temptable ] }]
view e view_name ( [( column_list] )]
as select_statement
[with [cascaded | local] check option]
```

or replace

可选项，用于指定 or replace 子句。该语句用于替换数据库中已有的同名视图，但需要在该视图上具有 DROP 权限

algorithm 子句

这个可选的 algorithm 子句是MySQL 对标注 SQL 的扩展，规定了 MySQL 处理视图的**算法**，这些算法会影响 MySQL 处理视图的方式。 Algorithm 可取三个值： undefined 、 merge 、temptable 。如果没有给出 Algorithm 的子句，则create view 语句的默认算法是 undefined （未定义的）。

merge

MySQL 首先将输入查询与定义视图的 SELECT 语句组合成单个查询。 然后 MySQL 执行组合查询返回结果集。 如果 SELECT 语句包含集合函数((如 MIN ， MAX ， SUM ， COUNT ， AVG 等)) 或 DISTINCT ， GROUP BY， HAVING ， LIMIT ， UNION ， UNION ALL ，子查询，则不允许使用MERGE 算法。 如果 SELECT 语句无引用表，则也不允许使用 MERGE 算法。如果不允许 MERGE 算法， MySQL 将算法更改为 UNDEFINED 。请注意，将视图定义中的**输入查询和查询组合成一个查询称为视图分辨率**。

temptable

MySQL 首先根据定义视图的 SELECT 语句创建一个临时表，然后针对该临时表执行输入查询。因为 MySQL 必须创建临时表来存储结果集并将数据从基表移动到临时表，所以 TEMPTABLE 算法的效率比MERGE 算法**效率低**。 另外，使用 TEMPTABLE 算法的视图是**不可更新的**

undefined

创建视图而不指定显式算法时， UNDEFINED 是**默认算法**。UNDEFINED 算法使 MySQL 可以选择使用 MERGE 或 TEMPTABLE 算法。MySQL 优先使用 MERGE 算法进行 TEMPTABLE 算法，因为 MERGE 算法效率更高。

view_name

指定视图的名称。该名称在数据库中必须是**唯一**的，不能与其他表或视图同名。

column_list

该可选子句可以为视图中的每个列指定明确的名称。其中列名的数目必须等于select 语句检索出来的结果数据集的列数，并且每个列名间用逗号分隔，如果省略 column_list 子句，则新建视图使用与基础表或源视图中相同的列名。

select_statement

用于指定创建视图的 select 语句，这个 select 语句给出了**视图的定义**，它可以用于查询多个基础表或者源视图。

with check option

该可选子句用于指定在可更新视图上所进行的修改都需要符合select_statement 中所指定的限制条件，这样可以确保数据修改后仍可以通过视图看到修改后的数据。当视图是根据另一个视图定义时， with check option
给出两个参数，即 cascaded 和 local ，它们决定检查测试的范围， **cascaded 为选项默认值**，会对所有视图
进行检查， **local** 则使 check option 只对定义的视图进行检查。

定义单源表视图

当视图的数据取自一个基本表的部分行、列，这样的视图称为单源表视图。此时视图的行列与基本表行列对应，用这种方法创建的视图可以对数据进行查询和修改操作。

eg

```
create view student_view1 as select * from student;
```

编写SQL语句，创建一个查询acc_transaction表中所有交易的交易历史编号、交易金额以及交易类型编码的视图,并用describe语句查看视图

```
CREATE VIEW acc_view1(TXN_ID, AMOUNT, TXN_TYPE_CD)
AS SELECT TXN_ID, AMOUNT, TXN_TYPE_CD
FROM acc_transaction;

DESCRIBE acc_view1;
```

查询视图acc_view1中交易金额大于等于10000并且小于等于50000的交易历史编号、交易类型编码

```
select atxn_id, a_txn_type_cd
from acc_view1
where a_amount <=50000 and a_amount>=10000;
```

选择视图acc_view1的前10行数据

```
 SELECT * FROM acc_view1 LIMIT 10;
```

定义多源表视图

多源表视图指定义视图的查询语句所涉及的表可以有多个，这样定义的视图一般只用于查询，不用于修改数据。

eg

在 student 、 course ， c sc 表上创建视图命名为scs_view ， 要求视图包含学生学号、姓名、课程名以及课程所对应的成绩及学分。

```
create view scs_view (sno,sname,cname,grade,credit)
as select sno,sname,cname,grade,credit
from student,course,sc
where student.sno=sc.sno and c.cno=sc.cno;
```

在已有视图上创建新视图

可以在视图上再创建视图，此时作为数据源的视图必须是已经建立好的视图。

eg

在刚才创建视图 scs_view 上创建一个只能浏览某一门课程成绩的视图，命名为 scs_view1 。

```
create view scs_view1
as
select *from scs_view
where scs_view.cname= ‘ MySQL 数据库设计 ’;
```

创建带表达式的视图

在定义基本表时，为减少数据库中的冗余数据，表中只存放基本数据，而基本数据经过各种计算派生出的数据一般是不存储的，但由于视图中的数据并不实际存储，所以定义视图时可以根据需要设置一些派生属性列，在这些派生属性列中保存经过计算的值。这些派生属性由于在基本表中并不实际存在，因此，也称它们为虚拟列。包含
虚拟列的视图也称为带表达式的视图。

eg

创建一个查询学生学号、姓名和出生年份的视图。

```
create view student_birthyear（sno,sname,birthyear）
as
select sno,sname,2023-sage
from student
```

含分组统计信息的视图

含分组统计信息的视图是指定义视图的查询语句中含有 group by 子句，这样的视图只能用于查询，不能用于修改数据

eg

创建一个查询每个学生的学号和考试平均成绩的视图

```
create view student_avg (sno,avggrade)
as
select sno,avg(grade) from sc
group by sno;
```

创建视图注意事项

权限

运行创建视图的语句需要用户具有创建视图（create view ）的权限，如果加上 [or replace]， 还需要用户具有删除视图（ drop view ）的权限

子查询

select 语句不能包含 from 子句中的子查询

系统或者用户变量

select 语句不能引用系统或者用户变量

预处理语句参数

select 语句不能引用预处理语句参数

创建 SQL 语句模板并发送到数据库。预留的值使用参数 "?" 标记 。例如：INSERT INTO MyGuests (firstname, lastname, email) VALUES(?, ?, ?)

在存储子程序内

在存储子程序内，定义不能引用子程序参数或者局部变量

引用的表/视图的存在问题

在定义中引用的表或者视图必须存在，但是创建了视图后，能够舍弃定义引用的表或者视图。要想检查视图定义是否存在这类问题，可以使用 check table 语句

temporary 表

在定义中不能引用 temporary 表，不能创建temporary 视图

在视图定义中命名的表必须已经存在

不能将触发程序与视图关联在一起

order by相关问题

在视图定义中允许使用 order by ， 但是，如果从特定视图进行选择，而该视图使用了具有自己 order by 的语句，它将被忽视

查看视图

查看视图是指查看数据库中已经存在的视图的定义。查看视图必须有 show view 的权限。查看视图的方法从不同的角度显示视图的相关信息。

describe 语句

```
describe view_name;
```

或

```
dec view_name;
```

show table status 语句

```
show table status like 'view_name';
```

show create view 语句

```
show create view 'view_name'
```

查看某数据库下的视图

查询 information_schema 数据库下的 view 表，语法格式为

```
select * from information_schema.views where table_name='view_name'
```

管理视图

修改视图

修改视图指修改数据库中已经存在表的定义。当基本表的某些字段发生改变时，可以通过修改视图来保持视图和基本表之间的一致。使用 alter view 语句用于修改一个先前创建好的视图，包括索引视图，但不影响相关的存储过程或触发器，也不更改权限。 alter view 语句语法格式为：

```
alter [algorithm={undefined |merge | e temptable ] }]
view e view_name ( [( column_list] )]
as select_statement
[with [cascaded | local] check option]
```

eg

使用alter view修改视图acc_view1的列名为交易历史编号、交易金额以及交易类型编码

```
ALTER VIEW acc_view1(交易历史编号,交易金额,交易类型编码)
AS SELECT TXN_ID, AMOUNT, TXN_TYPE_CD
FROM acc_transaction;
```

删除视图

在创建并使用视图后，如果确定不再需要某视图，或者想清除视图定义及与之相关的权限，可以使用 drop view 语句删除该视图，视图被删除后，基表的数据不受影响。

eg

```
drop view student_view2;
```

使用视图

使用视图查询数据

```
select * from view_name
```

使用视图更新数据

对视图的更新其实就是对表的更新，更新视图是指通过视图来插入（insert）、更新（update）和删除（delete）表中的数据

修改视图中的数据时，可以对基于两个以上基表或者视图的视图进行修改，但是不能同时影响两个或者多个基
表，每次修改都只能影响一个基表

不能修改那些通过计算得到的列，例如平均分等

如果创建视图时定义了 with check option 选项，那么使用视图修改基表中的数据时，必须保证修改后的数据满足定义视图的限制条件

执行 update 或者 delete 命令时，所更新或者删除的数据必须包含在视图的结果集中

如果视图引用多个表，使用 insert 或者 update 语句对视图进行操作时，被插入或更新的列必须属于同一个表

插入数据

可以通过视图向基表中插入数据，但插入的数据实际上存放在基表中，**而不在视图中**

更新数据

使用 update 语句可以通过视图修改基本表的数据

删除数据

使用 delete 语句可以通过视图删除基本表的数据

视图缺点

性能

从数据库视图查询数据可能会很**慢**，特别是如果视图是基于其他视图创建的

表依赖关系

根据数据库的基础表创建一个视图。每当更改与其相关联的表的结构时，都必须更改视图

MySQL 视图注意点

MySQL 5.x 版本之后支持数据库视图

不能在视图上创建索引

一个简单的视图可以更新表中数据。基于具有连接，子查询等的复杂 SELECT语句创建的视图无法更新

MySQL 不像 Oracle， PostgreSQL 等其他数据库系统那样支持物理视图， MySQL 是不支持物理视图的

存储过程（PROCEDURE）

存储过程（程序）和函数是事先经过编译并存储在数据库中的一套 SQL 语句

MySQL 存储过程的优点

提高性能

通常存储过程有助于提高应用程序的**性能**。当创建，存储过程被编译之后，就存储在数据库中。 但是，MySQL 实现的存储过程略有不同， MySQL 存储过程**按需编译**

减少流量

存储过程有助于减少应用程序和数据库服务器之间的**流量**，因为应用程序不必发送多个冗长的 SQL 语句，而只能发送存储过程的名称和参数

可重用和透明

存储的程序对任何应用程序都是**可重用的**和**透明的**。 存储过程将数据库接口暴露给所有应用程序，以便开发人员不必开发存储过程中已支持的功能

安全

存储的程序是安全的。 数据库管理员可以向访问数据库中存储过程的应用程序授予适当的权限，而不向基础数据库表提供任何权限

MySQL 存储过程的缺点

增加内存使用量

如果使用大量存储过程，那么使用这些存储过程的每个连接的内存使用量将会大大增加

很难调试存储过程

存储过程的构造使得开发具有复杂业务逻辑的存储过程变得更加困难。很难调试存储过程。只有少数数据库管理系统允许调试存储过程。**MySQL 不提供调试存储过程的功能**

开发、维护、移植困难

开发和维护存储过程并不容易，移植也比较困难

创建与修改存储过程

eg

返回单个数据

```
CREATE PROCEDURE qq_count (OUT num INT)
BEGIN
SELECT COUNT(*) INTO num FROM qq;
END;
```

返回数据集

```
CREATE PROCEDURE getrecord()
BEGIN
SELECT * FROM qq;
END;
```

存储过程基本语法

```
CREATE PROCEDURE sp_name ([proc_parameter[,...]])
[characteristic ...] routine_body
```

proc_parameter: [IN | OUT | INOUT] param_name type

characteristic:
LANGUAGE SQL
| [NOT] DETERMINISTIC
| { CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL DATA }
| SQL SECURITY { DEFINER | INVOKER }
| COMMENT 'string'

routine_body:
Valid SQL procedure statement or statements

存储过程说明

权限问题

创建存储子程序需要 CREATE ROUTINE 权限

修改或移除存储子程序需要 ALTER ROUTINE 权限。这个权限自动授予子程序的创建者

执行子程序需要 EXECUTE 权限。然而，这个权限自动授予子程序的创建者

关联问题

子程序与当前数据库关联。 要明确地把子程序与给定数据库关联起来，在创建子程序时指定其名字为 db_name.sp_name 。

当一个子程序被调用时，一个隐含的 USE db_name 被执行（当子程序终止时停止执行）。 存储**子程序内的 USE 语句是不允许的**

可以**使用数据库名限定子程序名。** 这可以被用来引用一个不在当前数据库中的子程序。比如，要引用一个与 test 数据库关联的存储程序p或函数f，可以 CALL test.p() 或 test.f()

数据库移除的时候，与它关联的所有存储子程序也都被移除

参数说明

由括号包围的参数列必须总是存在。 如果没有参数，也该使用一个空参数列 ( )

每个参数默认都是一个 IN 参数。 要指定为其它参数，可在参数名之前使用关键词 OUT 或 INOUT

IN 输入参数：表示调用者向过程传入值（传入值可以是字面量或变量）

OUT 输出参数：表示过程向调用者传出值((可以返回多个值)) （传出值只能是变量）

INOUT 输入输出参数：既表示调用者向过程传入值，又表示过程向调用者传出值（值只能是变量）

Characteristic 特性说明

CONTAINS SQL 表示子程序包含 SQL 但不包含读或写数据的语句

NO SQL 表示子程序不包含 SQL 语句

READS SQL DATA 表示子程序包含读数据的语句，但不包含写数据的语句

MODIFIES SQL DATA 表示子程序包含写数据的语句

默认的是CONTAINS SQL

SQL SECURITY 特征可以用来指定子程序该用创建子程序者的许可来执行，还是使用调用者的许可来执行。 **默认值是 DEFINER**

COMMENT 子句是一个 MySQL 的扩展，它可以被用来描述存储程序，被SHOW CREATE ROCEDURE 和 SHOW CREATE FUNCTION显示

过程体说明

*MySQL 允许子程序包含 DDL 语句，如 CREATE 和 DROP 。 MySQL 也允许**存储程序（但不是存储函数）**包含 SQL 交互 select*

存储函数不可以包含那些做明确的和绝对的提交或者做回滚的语句

存储子程序不能使用**LOAD DATA INFILE**

返回结果包的语句不能被用在存储函数中

其它语句：块语句、选择、循环等等

BEGIN ... END 复合语句块

```
[begin_label:] BEGIN
[statement_list]
END [end_label]
```

存储子程序可以使用 **BEGIN ... END 来包含多个语句**

statement_list 代表一个或多个语句的列表， **每个语句都必须用分号（；）来结尾**

复合语句可以被标记。 除非 begin_label 存在, , 否则 end_label 不能被给出, 并且如果二者都存在, , 他们必须是同样的。

修改和删除存储过程

修改语法

```
ALTER PROCEDURE sp_name [characteristic ...]
characteristic:
{ CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL
DATA }
| SQL SECURITY { DEFINER | INVOKER }
| COMMENT 'string'
```

说明： 如果要重新完整的定义已有的存储过程，建议采用先删除该存储过程后，然后再进行创建。

删除语法

```
DROP PROCEDURE [IF EXISTS] sp_name
```

IF EXISTS 子句是一个 MySQL 的扩展。 如果程序或函数不存储，它防止发生错误

查看存储过程

SHOW CREATE {PROCEDURE | FUNCTION} sp_name

eg

```
SHOW CREATE PROCEDURE getrecord;
```

SHOW {PROCEDURE | FUNCTION} STATUS [LIKE 'pattern']

eg

```
SHOW PROCEDURE status like 'getrecord'
```

查看系统表information_schema.Routines

eg

```
SELECT * FORM Routines where routine_name= = 'getrecord'
```

使用变量

使用变量的基本步骤

1 DECLARE局部变量

```
DECLARE var_name [,...] type [DEFAULT value]
```

DECLARE 仅被用在 BEGIN ... END 复合语句里，并且必须在复合语句的开头，在任何其它语句之前。

DEFAULT 子句指定默认值（常量或表达式），不指定则初始值为NULL

2 SET语句为变量复制

```
SET e var_name = expr [, e var_name = expr] ...
```

SET 语句可以同时给多个变量赋值

3 SELECT ... INTO 语句

```
SELECT col_name [,...] INTO var_name [,...] table_expr
```

把选定的多个字段直接存储到变量

只有一条记录的字段可以被取回

eg

```
CREATE PROCEDURE qq_t_var) ()
begin
declare id int;#1
declare name varchar(10);#1
declare mail varchar(25);#1
set id=1;#2
select tid,qqname,email into l id,name,mail from qq_t where tid =id;#3
select id,name,mail;#3
```

流程控制语句

IF语句

```
IF expression THEN
    statements;
END IF;
```

```
IF expression THEN
    statements;
ELSE else-statements;
END IF;
```

```
IF expression THEN
    statements;
ELSEIF expression THEN
    statements;
ELSE else-statements;
END IF;
```

CASE 语句

```
CASE case_expression
    WHEN when_expression_1 THEN commands
    WHEN when_expression_2 THEN commands
    ...
    ELSE commands
END CASE;
```

简单CASE 语句比 IF语句更有效率

选择使用 CASE 语句，则必须确保至少有一个 CASE 条件匹配。否则，需要定义一个错误处理程序来捕获错误

WHILE循环

```
WHILE expression DO
    statements
END WHILE
```

REPEAT 循环

```
REPEAT
    statements;
UNTIL expression
END REPEAT
```

LOOP,LEAVE 和 ITERATE 语句

LEAVE == BREAK

ITERATE == CONTINUE

LOOP:可以反复执行一个代码块，可以使用循环标签达到灵活性

定义条件和处理

定义条件和处理程序 能够事先定义程序执行过程中有可能遇到的问题，并采用一定的机制解决相关问题： 继续或退出当前代码块的执行，并发出有意义的错误消息

基本步骤

1 DECLARE 条件的定义

```
DECLARE condition_name CONDITION FOR condition_value
```

condition_name： 条件的名称

condition_value： 可以取 sqlstate_value 或 mysql_error_code， 都表示MYSQL 的错误代码

eg

重复主键值的错误代码为 ERROR 1062(23000)

```
sqlstate_value =23000 ， mysql_error_code
```

2 条件的处理

```
DECLARE handler_type HANDLER FOR condition_value [,...] sp_statement
```

handler_type: : | CONTINUE | EXIT | UNDO（不支持）

CONTINUE： 继续执行封闭代码块（BEGIN...EDN）

EXIT：处理程序声明封闭代码块的执行终止

condition_value:sqlstate_value | condition_name | SQLWARNING | NOT FOUND | SQLEXCEPTION | mysql_error_code

SQLWARNING 是对所有以 01 开头的 SQLSTATE 代码的速记

NOT FOUND 是对所有以 02 开头的 SQLSTATE 代码的速记

SQLEXCEPTION 是对所有没有被 SQLWARNING 或 NOT FOUND捕获的 SQLSTATE 代码的速记

eg

```
CREATE PROCEDURE qq_condition ()
BEGIN
DECLARE CONTINUE HANDLER FOR SQLSTATE '23000' SET @x2 = 1;
SET @x = 1;
INSERT INTO qq(qqno,tid) VALUES ('68747539',1);
SET @x = 2;
INSERT INTO qq(qqno,tid) VALUES ('705646790',1);
SET @x = 3;
END;
```

```
DECLARE EXIT HANDLER FOR SQLSTATE '23000' SET @x2 = 1;
```

其他条件形式

捕获 mysql_error_code
DECLARE CONTINUE HANDLER FOR 1062 SET @x2 = 1;

事先定义 condition_name
DECLARE DuplicateKey CONDITION FOR SQLSTATE '23000';
DECLARE CONTINUE HANDLER FOR y DuplicateKey SET @x2 = 1;

捕获 SQLEXCEPTION
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION SET @x2 = 1;

存储过程示例

创建并执行存储过程acc_process,创建一个视图acc_type，并选择该视图的前10行数据。 acc_type包含两个列：账户编号(ACCOUNT_ID)和账户属性(ACCOUNT_TYPE)

```
DROP PROCEDURE IF EXISTS acc_process;
CREATE PROCEDURE acc_process()
BEGIN
       DROP VIEW IF EXISTS acc_type;
       CREATE VIEW acc_type(ACCOUNT_ID, ACCOUNT_TYPE)
       AS SELECT account.ACCOUNT_ID, get_acc_type(account.AVAIL_BALANCE)
       FROM account ;
       SELECT * FROM acc_type LIMIT 10;##! !
END;
CALL acc_process;##! !
```

创建存储过程updateCloseDate，该存储过程更新account表中的关闭日期（CLOSE_DATE），根据产品编号做不同的更新操作，要求使用游标：

1. 对产品编号对应类型为存款的账户（即产品编号PRODUCT_CD对应PRODUCT_TYPE_CD为ACCOUNT），如开户日期（OPEN_DATE）在2015-01-01之前（含）的，设置其关闭日期（CLOSE_DATE）为开户日期加20年，否则为开户日期加30年；
2. 对产品编号对应类型为贷款的账户（即产品编号PRODUCT_CD对应PRODUCT_TYPE_CD为LOAN），如可用余额（AVAIL_BALANCE）少于100000的（含），设置其关闭日期（CLOSE_DATE）为开户日期加20年，否则为开户日期加30年；
3. 对产品编号对应类型为保险的账户（即产品编号PRODUCT_CD对应PRODUCT_TYPE_CD为INSURANCE），设置其关闭日期（CLOSE_DATE）为开户日期加15年。

```
DROP PROCEDURE IF EXISTS updateCloseDate;
CREATE PROCEDURE updateCloseDate ()
BEGIN
    DECLARE id INT;
    DECLARE type VARCHAR(255);
    DECLARE opend DATE;
    DECLARE closed DATE;
    DECLARE bal DECIMAL ( 12, 4 );
    DECLARE cur CURSOR FOR
    SELECT
```

```
        a.ACCOUNT_ID,
        p.PRODUCT_TYPE_CD,
        a.OPEN_DATE,
        a.CLOSE_DATE,
        a.AVAIL_BALANCE
    FROM
        account a,
        product p
    WHERE
        a.PRODUCT_CD = p.PRODUCT_CD;
    DECLARE EXIT HANDLER FOR NOT FOUND CLOSE cur;
    OPEN cur;
    REPEAT
            FETCH cur INTO id, type, opend, closed, bal;
        CASE type
            WHEN 'ACCOUNT' THEN
                UPDATE account
                SET CLOSE_DATE = DATE_ADD(opend, INTERVAL IF (opend <= '2015-01-01',
20, 30)YEAR)
                WHERE ACCOUNT_ID = id;
            WHEN 'LOAN' THEN
                UPDATE account
                SET CLOSE_DATE = DATE_ADD( opend, INTERVAL IF ( bal <= 100000, 20,
30 ) YEAR )
                WHERE ACCOUNT_ID = id;
            WHEN 'INSURANCE' THEN
                UPDATE account
                SET CLOSE_DATE = DATE_ADD( opend, INTERVAL 15 YEAR )
                WHERE ACCOUNT_ID = id;
        END CASE;
        UNTIL 0
    END REPEAT;
    CLOSE cur;
END;
```

函数（FUNCTION）

创建与修改函数

```
CREATE FUNCTION id_email (id int) RETURNS varchar(20)
Reads SQL data
BEGIN
declare email varchar(20);
SELECT email INTO email FROM qq where tid =id;
RETURN email;
END;
```

函数基本语法

```
CREATE FUNCTION sp_name ([func_parameter[,...]]) RETURNS type
[characteristic ...] routine_body
```

func_parameter: param_name type(Any valid MySQL data type)

RETURNS 字句只能对 FUNCTION 做指定，对函数而言这是强制的。它用来指定函数的返回类型，而且**函数体必须包含一个 RETURN value**

函数示例

创建函数emp_cnt：以支行名称(branch.NAME)为输入参数，返回该支行员工总人数

并利用上述函数，查询上海市内每间支行的员工总数

```
DROP FUNCTION IF EXISTS emp_cnt;
CREATE FUNCTION emp_cnt(brh VARCHAR(20)) RETURNS int
Reads SQL data########!!
BEGIN
       DECLARE cnt INT;
       SELECT COUNT(employee.EMP_ID) INTO cnt FROM employee, branch
       WHERE employee.ASSIGNED_BRANCH_ID = branch.BRANCH_ID and branch.NAME = brh;
       RETURN cnt;
END;
==========================================================================================
SELECT NAME, emp_cnt(NAME)
FROM branch WHERE CITY = '上海市';
```

创建函数get_acc_type,以账户余额(account.AVAIL_BALANCE为输入，根据该账户的余额返回该账户的类型。类型划分标准为：若余额大于100000，账户类型为'P1'若余额大于10000，账户类型为'P2'否则账户类型为'P3'

```
DROP FUNCTION IF EXISTS get_acc_type;
CREATE FUNCTION get_acc_type(thebalance int(32))RETURNS VARCHAR(20)
READS SQL DATA
BEGIN
IF (thebalance > 100000) THEN
RETURN 'P1';
ELSEIF (thebalance >10000) THEN
RETURN 'P2';
ELSE
RETURN 'P3';
END IF;#####!!
END;
```

修改和删除函数

修改语法

```
ALTER FUNCTION sp_name [characteristic ...]
characteristic:
{ CONTAINS SQL | NO SQL | READS SQL DATA | MODIFIES SQL
DATA }
| SQL SECURITY { DEFINER | INVOKER }
| COMMENT 'string'
```

说明： 如果要重新完整的定义已有的存储过程，建议采用先删除该存储过程后，然后再进行创建。

删除语法

```
DROP FUNCTION [IF EXISTS] sp_name
```

参考存储过程

游标

定义

游标从本质上讲是系统为用户开设的一个数据缓冲区，用于存放T - SQL 语句从数据库检索出来的结果集

当应用程序需要对结果集进行处理时，使用游标可以从结果集中一条条地提取记录，为此每个游标必须有一个名字

游标允许应用程序对查询语句 select 返回的行结果集中每一行进行相同或不同的操作，而不是一次对整个结果集进行同一种操作

MySQL 游标为只读，不可滚动和敏感

只读：无法通过游标更新基础表中的数据

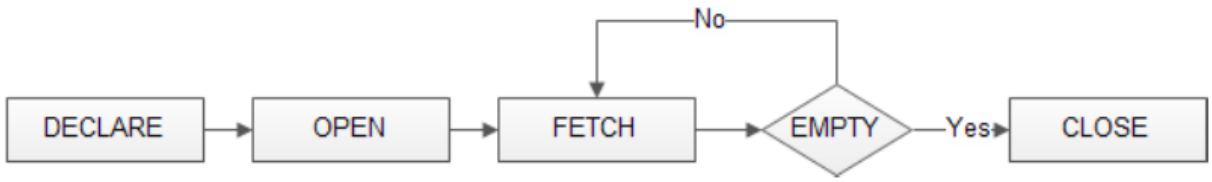
不可滚动：只能按照 SELECT 语句确定的顺序获取行。不能以相反的顺序获取行。 此外，不能跳过行或跳转到结果集中的特定行

敏感游标指向实际数据，不敏感游标使用数据的临时副本

敏感游标：快，对其他连接的数据所做的任何更改都将影响由敏感游标使用的数据，如果不更新敏感游标所使用的数据，则更安全

使用游标

利用基于变量的 select into 语句，仅能处理一条记录的数据。通过游标（或光标），能够对查询的结果集进行循环处理



基本步骤

1.声明游标

```
DECLARE e cursor_name CURSOR FOR select_statement
```

2.打开游标

```
OPEN cursor_name
```

3.使用游标

```
FETCH cursor_name INTO e var_name [, var_name ] ...
```

4.关闭游标

```
CLOSE cursor_name
```

说明

1 、 游标必须在声明处理程序之前被声明，并且声明变量和条件之后
2 、 SELECT 语句中不能有 INTO 子句

游标示例

利用游标，创建一存储过程实现将 qq_t 表中的数据添加到 qq 表中

```
CREATE PROCEDURE qq_cursor_insert ()
BEGIN
DECLARE a varchar(15);
DECLARE b int;
DECLARE c varchar(10);
DECLARE d varchar(25);
DECLARE cur1 CURSOR FOR SELECT qqno,tid,qqname,email FROM qq_t;
DECLARE exit HANDLER FOR NOT FOUND CLOSE cur1;
OPEN cur1;
REPEAT
FETCH cur1 INTO a,b,c,d;
insert into qq(qqno,tid,qqname,email) values(a,b,c,d);
UNTIL 0 END REPEAT;
CLOSE cur1;
END
```

触发器

定义

触发器定义了一系列操作，这一系列操作称为触发程序，当触发事件发生时，触发程序会自动运行

触发器 主要用于监视某个表的插入 （ insert ） 、更新 （ update ） 和删除 （ delete ） 等更新操作，这些操作可以分别激活该表的 insert 、 update 和 delete 类型的触发程序运行，从而实现数据的**自动维护**。

触发器的使用

安全性

可以基于数据库的值使用户具有操作数据库的某种权利。可以基于**时间**限制用户的操作，例如，不允许下班后和节假日修改数据库数据等。可以基于数据库中的**数据**限制用户的操作，例如，不允许学生的分数大于满分等

审计

可以 跟踪用户对数据库的操作。审计用户操作数据库的语句，把用户对数据库更新写入审计表

实现复杂的数据完整性规则

实现非标准的数**据完整性检查和约**束。触发器可产生比规则更为复杂的限制。与规则不同，触发器可以引用列或者数据库对象。例如，触发器可回退任何企图吃进超过自己保证金的期货

实现复杂的非标准的数据库相关完整性规则

在修改或者删除时，级联修改或者删除企图表中与之匹配的行。在修改或者删除时把其他表中与之匹配的行设成 null 值。在修改或者删除时把其他的表中与之匹配的行级联设成默认值。触发器能够拒绝或者回退破坏相关完整性的变化，取消试图进行数据更新的事务。当插入一个与其主键不匹配的外键时，这种触发器起作用。

创建并使用触发器

创建触发器

触发程序是与表有关的命名数据库对象，当表上出现特定事件时，将激活该对象在 MySQL 中，可以使用 create trigger语句创建触发器，具体语法格式为：

```
create trigger e trigger_name e trigger_time trigger_event
on tbl_name for each row trigger_stmt
```

trigger_name：触发器的名称，触发器在当前数据库中必须具有**唯一性名称**。如果要在某个特定数据库中创建，名称前面应该加上数据库的名称

trigger_time： trigger_time 是触发器被触发的时间。 它可以是 before 或者 after

trigger_event：指明了激活触发器的语句的类型

tbl_name：与触发器相关联的表名。 tbl_name 必须引用**永久性表**。不能将触发程序与 temporary 表或视图关联起来 。在该表上触发事件发生时才会激活触发器，同一个表不能拥有两个具有相同触发时刻和事件的触发器

for each row： 用来指定对于受触发事件影响的每一行都要激活触发器的动作

trigger_stmt：是当**触发程序激活时执行的语句。**如果你打算执行多个语句，可使用 **begin... end** 复合语句结构。这样，就能使用存储子程序中允许的相同语句

使用触发器

eg

创建并使用触发器实现检查约束，保证课程的人数上限 up_limit 字段值在（ 60,150,230 ）范围内

```
create trigger course_insert_before_trigger before insert
on course for each row
begin
if( new.up_limit =60|| new.up_limit =150|| new.up_limit =230) then
set new.up_limit= = new.up_limit;
else insert into mytable valuses(0);
end if;
end;
```

-使用old/new区分新旧值

-INSERT仅有NEW.col_name(**REPLACE, LOAD DATA均可触发**)

-DELETE仅有OLD.col_name

-UPDATE均有

-OLD readonly

在前例之上，创建 course_update_before_trigger 触发器，负责进行修改检查

```
create trigger r course_update_before_trigger before insert
on course for each row
begin
if( new.up_limit !=60|| new.up_limit !=150|| new.up_limit !=230) then
set new.up_limit= = old.up_limit;
end if;
end;
```

查看触发器

查看触发器指查看数据库中已经存在的触发器的定义、权限和字符集等信息，可以使用下面4 4 种方法查看触发器的定义

show triggers

使用“ show trigger \ G” 命令可以查看当前数据库中所有触发器的信息，用这种方式查看触发器的定义时，可以查看当前数据库中所有触发器的定义。如果触发器太多，可以使用“ show trigger like 模式\ G” 命令查看与模式模糊匹配的触发器信息。

information_schema.triggers

MySQL 中所有触发器的定义都存放在 **information_schema 数据库下的triggers 表**中，查询 triggers 表时，可以查看数据库中所有触发器的详细信息，查询语句如下:

```
select *from information_schema.triggers\G
```

show create trigger

使用“ show create trigger”命令可以查看某一个触发器的使用

记事本

成功创建触发器后， MySQL 自动在数据库目标下创建 TRN 以及 TRG触发器文件，以记事本方式打开文件可以查看触发器的定义

删除触发器

```
drop trigger[schema_name].trigger_name
```

schema_name：可选项，用于指定触发器所在的数据库的名称。如果没有指定，则为当前默认数据库

trigger_name ： 要删除的触发器名称

drop trigger 语句需要 super 权限

当删除一个表的同时，也会自动删除表上的触发器。另外，**触发器不能更新或者覆盖**，为了修改一个触发器，必须先删除它，然后再重新创建

触发器的应用

触发器使用注意事项

在 MySQL 中使用触发器时有一些注意事项

-如果触发程序中包含select 语句， select 语句不能返回结果集。

-同一个表不能创建两个相同触发时间、触发事件的触发程序（*5.7.2 版本之后可以*）。

-触发程序中*不能*使用以显示或者隐式方式打开、开始或者结束**事务**的语句，如 start transaction 、 commit 、 rollback 或者set autocommit =0 语句。

-MySQL 触发器针对记录进行操作，当批量更新数据时，引入触发器会导致批量更新操作的性能降低

-在 MySQL 存储引擎中，触发器不能保证原子性，例如，当使用一个更新语句更新一个表后，触发程序实现另外一个表的更新，如果触发程序执行失败，那么不会回滚第一个表的更新。 InnoDB 存储引擎支持事务，使用触发器可以保证更新操作与触发程序的原子性，此时触发程序和更新操作是在同一个事务中完成的，

-InnoDB 存储引擎实现外键约束关系时，建议使用级联选项维护外键数据；使用触发器维护 InnoDB 外键约束的级联选项时，应该首先维护子表的数据，然后再维护父表的数据，否则可能出现错误

-MySQL 的触发程序不能对本表执行 update 操作，触发程序中的 update 操作可以直接使用 **set** 命令替代，否则可能出现错误，甚至陷入**死循环**

-在 before 触发程序中， auto_increament 字段的 new 值为0，不是实际插入新纪录时自动生成的自增型字段值

-添加触发器后，建议对其进行详细的测试，测试通过后再决定是否使用触发器

使用触发器的例子

维护冗余数据

冗余的数据需要额外的维护，维护冗余数据时，为了**避免数据不一致问题**的发生（例如，剩余的学生名额+ +已选学生人数 课程的人数上限），冗余数据应该尽量避免交由人工维护，建议交由应用系统（例如触发器）自动维护

eg

某学生选修了某门课程，请创建choose_insert_before_trigger 触发器维护课程 available 的字段值。

```
create trigger choose_insert_before_trigger before insert
on choose for each r r ow
begin
update course set available=available- - 1 where course_no=new.course_no;
end;
```

某学生放弃选修某门课程，请创建choose_delete_before_trigger 触发器维护课程 available 的字段值

```
create trigger choose_delete_before_trigger before insert
on choose for each r r ow
begin
update course set available=available+1 where course_no=old.course_no;
end;
```

使用触发器模拟外键级联选项

-对于 InnoDB 存储引擎的表而言，由于支持外键约束，在定义外键约束时，通过设置外键的级联选项 cascade 、set null 或者 no action （ restrict ），外键约束关系可以交由InnoDB 存储引擎自动维护

-如果 InnoDB 存储引擎的表之间存在外键约束关系，但是不存在级联选项；或者使用数据库表为 MyISAM （ 该 表不支持外键约束关系），则可以使用触发器**模拟实现**“外键约束”之间的“级联选项”

eg

在删除分类时自动删除分类对应的消息记录

```
create trigger choose_delete_before before delete
on mtypre for each row
begin
delete from message where typeId = old.id
end;
```

触发器示例

先在account表中执行更新语句，将最后活跃日期和当前时间相距超过三年的所有账户的Status属性设置为“不活跃”。 UPDATE account a SET a.`STATUS` = '不活跃' WHERE DATE_ADD(a.LAST_ACTIVITY_DATE, INTERVAL 3 YEAR) < CURDATE(); 然后，在acc_transaction上定义一个AFTER INSERT触发器，触发器名为“d_newTransaction”，当对acc_transaction表执行任何插入操作后被启动，将该账户在account表中的Status属性设置为 “正常”。

```
DROP TRIGGER IF EXISTS d_newTransaction;
CREATE TRIGGER d_newTransaction AFTER INSERT ON acc_transaction FOR EACH ROW
BEGIN
UPDATE account a
SET a.`STATUS` = '正常'
WHERE a.ACCOUNT_ID = new.ACCOUNT_ID;
END;
```

在acc_transaction上定义触发器t_newTransaction，当往acc_transaction中插入一条数据时，依据账户编号（ACCOUNT_ID）更新account表中对应账户的可用余额（AVAIL_BALANCE）和最后活跃日期（LAST_ACTIVITY_DATE）：

1. 如果插入数据的交易类型编码（TXN_TYPE_CD）为CD、TT、IC、LI则设置可用余额为当前可用余额加上交易金额、最后活跃日期为当前日期
2. 如果插入数据的交易类型编码（TXN_TYPE_CD）为CW、TF则设置可用余额为当前可用余额减去交易金额、最后活跃日期为当前日期；如当前可用余额减去交易金额小于0，则撤销对 acc_transaction此条数据的插入

```
DROP TRIGGER IF EXISTS t_newTransaction;
CREATE TRIGGER t_newTransaction BEFORE INSERT ON acc_transaction FOR EACH ROW
BEGIN
    DECLARE bal INT;
    IF
        new.TXN_TYPE_CD = 'CW' OR new.TXN_TYPE_CD = 'TF'
        THEN
            SELECT a.AVAIL_BALANCE INTO bal
            FROM account a
            WHERE a.ACCOUNT_ID = new.ACCOUNT_ID;
        IF
            bal < new.AMOUNT
        THEN
            DELETE FROM acc_transaction
            WHERE TXN_ID = new.TXN_ID;
        ELSE UPDATE account a SET a.AVAIL_BALANCE = a.AVAIL_BALANCE - new.AMOUNT,
            a.LAST_ACTIVITY_DATE = DATE(new.TXN_DATE) WHERE a.ACCOUNT_ID =
new.ACCOUNT_ID;
        END IF;
    ELSE UPDATE account a SET a.AVAIL_BALANCE = a.AVAIL_BALANCE + new.AMOUNT,
a.LAST_ACTIVITY_DATE = DATE(new.TXN_DATE) WHERE a.ACCOUNT_ID = new.ACCOUNT_ID;
    END IF;
END;
```

请在players表中创建如下两个触发器：

1. before触发器：插入player表之前，检测该条插入数据的salary是不是小于10,000，如果是则增大10倍；
2. after触发器：每添加一条数据，就更新team表中nums值(该队下的总人数)。

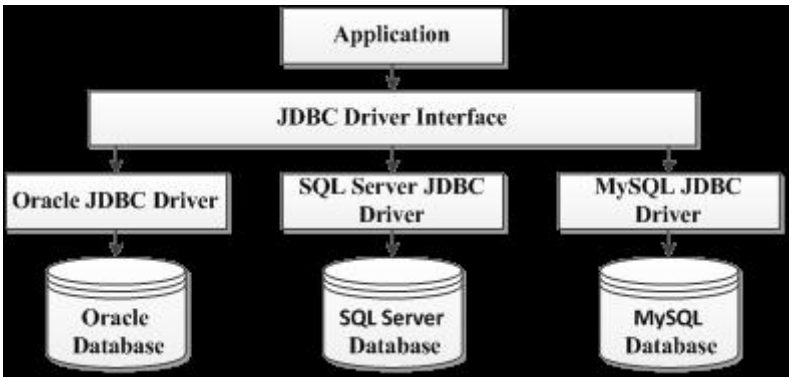
```
1.

DROP TRIGGER IF EXISTS `before_inseret`;
CREATE TRIGGER `before_inseret` BEFORE INSERT ON `players` FOR EACH ROW
BEGIN
    IF
        new.salary < 10000 THEN
        SET new.salary = 10 * new.salary;
    END IF;
END;
2.
DROP TRIGGER IF EXISTS `insert_player`;
CREATE TRIGGER `insert_player` AFTER INSERT ON `players` FOR EACH ROW
BEGIN
    UPDATE teams
    SET nums = nums + 1
    WHERE teams.id = new.team_id;
END;
```

JDBC连接数据库

JDBC 概述

JDBC （Java DataBase Connectivity， Java 数据库连接）是一种用于执行 SQL 语句的 Java API （Application Programming Interface， 应用程序接口），可以为多种关系型数据库提供统一访问。



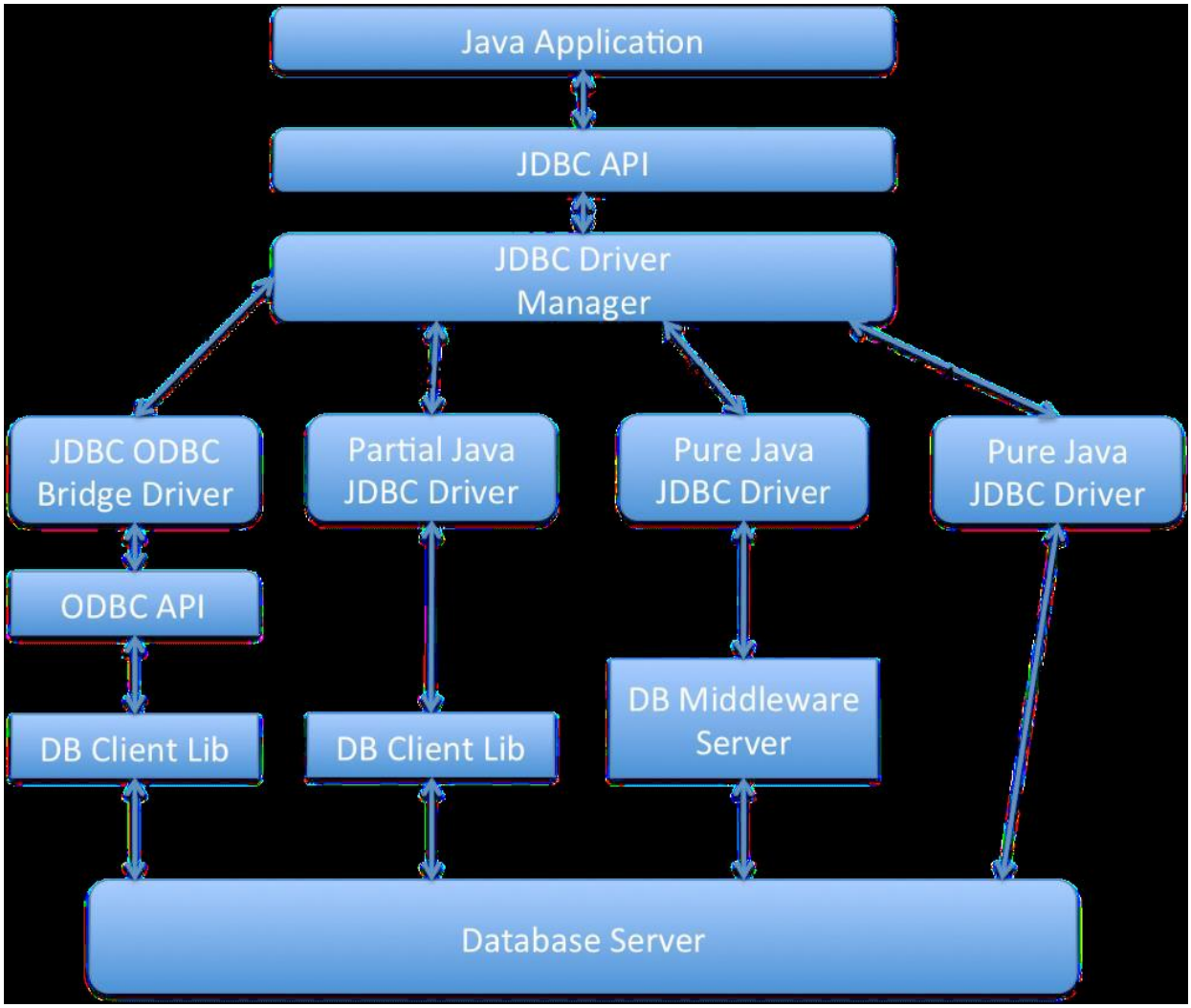
JDBC 的基本功能

（1）建立与数据库的连接；

（2）向数据库发送 SQL 语句；

（3）处理从数据库返回的结果；

常见的 JDBC 驱动程序可分为4种类型



Type1:JDBC- ODBC Bridge driver

Type2:native- API,partly Java driver

Type3:JDBC-Net pure Java driver

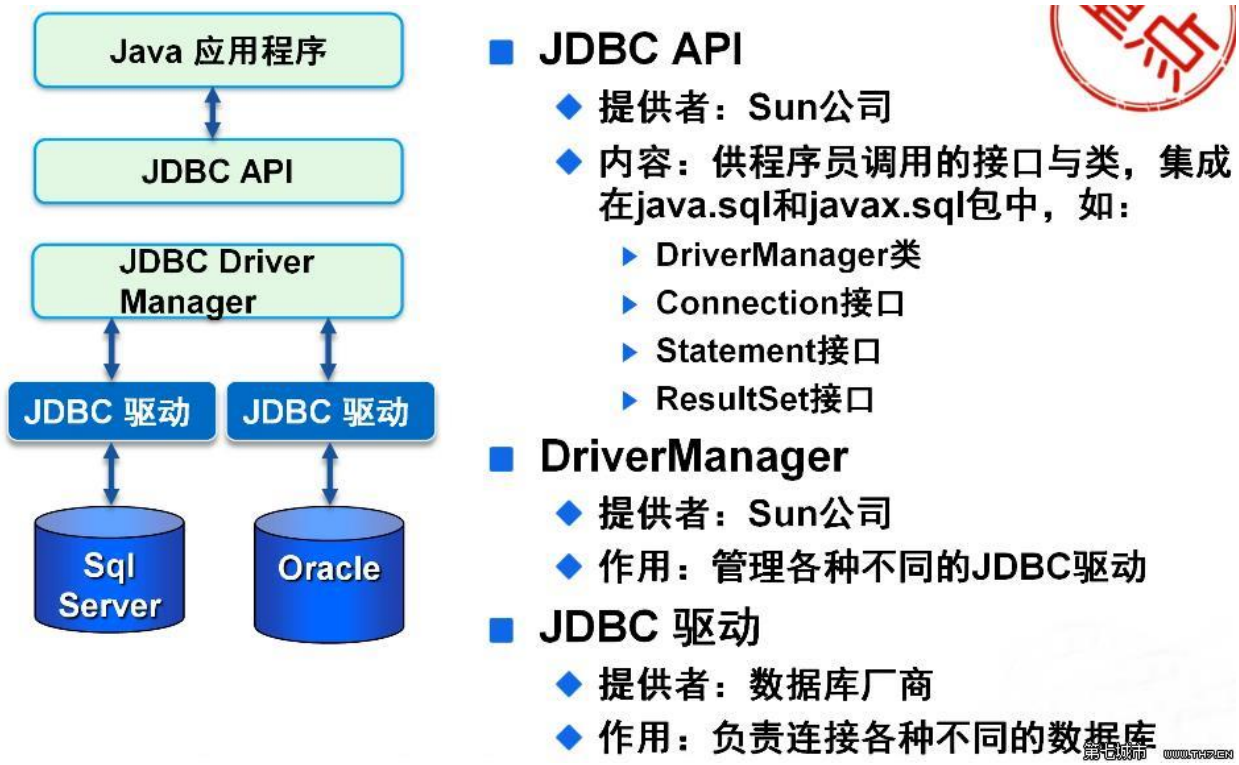
Type4:native protocol, pure Java driver

JDBC 连接过程

JDBC 是一种底层 API， 不能直接访问数据库

要通过 JDBC 来存取某一特定的数据库，必须依赖于相应数据库厂商提供的 JDBC 驱动程序

JDBC 驱动程序是连接 JDBC API 与具体数据库之间的桥梁



下载的 JDBC 驱动包，对于普通的 Java Application应用程序，只需要将 JDBC

对于 Java Web 应用，通常将 JDBC 驱动包放置在项目的 WEB- - INF/lib 目录下

MySQL Connector/J

MySQL Connector/J 是一个 **JDBC Type 4** 驱动程序，实现了 JDBC 4.2 规范

JDBC 连接 MySQL 数据库步骤：

（1）加载 JDBC 驱动程序；
（2）创建数据库连接；
（3）创建 Statement 对象；
（4）执行 SQL 语句；
（5）处理执行 SQL 语句的返回结果；
（6）关闭连接。

加载 JDBC 驱动程序

-加载驱动程序的方法是使用 java.lang.Class 类的静态方法forName (String className)) 。
Class.forName(" com.mysql.cj.jdbc.Driver")

-若加载成功，系统将加载的驱动程序注册到DriverManager 类中。如果加载失败，将抛出 **ClassNotFoundException** 异常，即未找到指定的驱动类

```
try {
e Class.forName ("com.mysql.cj.jdbc.Driver");
} catch (ClassNotFoundException ex) {
System.out.println(" 加载数据库驱动时抛出异常!");
ex.printStackTrace();
} }
```

创建数据库连接

-Connection 接口代表与数据库的连接，只有建立了连接，用户程序才能操作数据库

-一个应用程序可与单个数据库有一个或多个连接，也可以与多个数据库有连接

-与数据库建立连接的方法是调用 **DriverManager** 类的**getConnection()** 方法

-getConnection() 方法的返回值类型为 java.sql.Connection ，如果连接数据库失败，将抛出 **SQLException** 异常

方法调用

```
Connection conn = DriverManager.getConnection(String url, String userName, String
password);
```

依次指定要连接数据库的路径、用户名及密码，即可创建数据库连接对象

eg

若要连接 BookStore 数据库，则连接数据库路径的写法如下：

```
jdbc:mysql//localhost:3306/bookstore
```

也可以采用带数据库数据编码格式的方式

```
jdbc:mysql ://localhost:3306/ bookstore?useUnicode= = true&chara
cterEncoding
```

连接 BookStore

```
try {
String url ="jdbc:mysql://localhost:3306/bookstore";
String user="root"; // 访问数据库的用户名
String password="123456"; // 访问数据库的密码
Connection conn= DriverManager.getConnection(url,user,password);
System.out.println(" 连接数据库成功！");
} catch (SQLException ex) {
System.out.println(" 连接数据库失败 ");
ex.printStackTrace();
} }
```

创建 Statement 对象

连接数据库后，要执行 SQL 语句，必须创建一个 Statement 对象

（1）创建 Statement 对象
（2）创建 PreparedStatement 对象

创建 Statement 对象

利用 Connection 接口的 createStatement) () 方法可以创建 Statement 对象，用来执行**静态**的 SQL 语句

```
Statement stmt= = conn.createStatement(); //conn为数据库连接对象
```

创建 PreparedStatement 对象

利用 Connection 接口的 prepareStatement (String sql)) 方法可以创建 PreparedStatement 对象，用来执行**动态**的 SQL 语句

创建 PreparedStatement 对象 pstmt

```
String l sql = "select * from users where u_id >? and u_sex =?";
PreparedStatement t pstmt = conn.prepareStatement( sql );
```

对每个输入参数进行设置

```
pstmt.setXxx(position,value);
```

eg

```
pstmt.setInt (1,3);
pstmt.setString (2," 女);
```

执行 SQL 语句

创建 Statement 对象后，就可以利用该对象的相应方法来执行 SQL 语句，实现对数据库的具体操作

Statement 对象的常用方法有 executeQuery() 、executeUpdate()

ResultSet executeQuery(String sql) 方法：该方法用于执行产生**单个结果集**的 SQL 语句，如 SELECT 语句，该方法返回一个结果集 ResultSet 对象

int executeUpdate(String sql) 方法：该方法用于执行 INSERT 、UPDATE 或 或 DELETE 语句以及 SQL DDL（数据定义语言）语句。该方法的返回值是一个整数，表示**受影响的行数**。对于 于 CREATE TABLE 或 或 DROP TABLE 等不操作数据行的语句，返回值为0。

将查询结果保存到 ResultSet 对象 rs 中

```
String sql = "select * from users";
ResultSet rs= stmt.executeQuery(sql);
```

eg

删除 users 表中 u_id 为7 的记录

```
String sql = "delete from users where u_id=7";
int rows= stmt.executeUpdate(sql);
```

PreparedStatement 对象也可以调用 executeQuery() 和executeUpdate() 两个方法，但都不需要带参数。

要删除 users 表中 u_id 为7的记录

```
String sql = "delete from users where u_id=?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setInt(1,7);
int rows= pstmt.executeUpdate ();
```

处理执行 SQL 语句的返回结果

使用 Statement 对象的 executeQuery) () 方法执行一条SELECT 语句后，会返回一个 ResultSet 对象

ResultSet 对象保存查询的结果集，调用 ResultSet 对象的相应方法就可以对结果集中的数据行进行处理

ResultSet 对象的常用方法

nboolean next()： ResultSet 对象具有指向当前数据行的**指针**，指针最初指向第一行之前，使用 next() 方法可以将指针移动到下一行。如果没有下一行时，则返回 false

getXxx((列名或列索引))：该方法可获取所在行指定列的值。其中， Xxx 指的是列的数据类型。若使用列名作为参数，则getString (“name”) ，表示获取当前行列名为“ name” 的列值。列索引值**从1开始编号**，如第2 2列对应的索引值为2 。

获取 users 表中第一条记录的基本信息

```
String l sql = "select * from users";
ResultSet rs =stmt. executeQuery(sql);
rs.next ();
int u_id= = rs.getInt (1);
// 或int u_id= = rs.getInt" ("u_id");
String u_name= = rs.getString (2);
String u_sex= = rs.getString" ("u_sex");
```

关闭连接

连接数据库过程中创建的 Connection 对象、 Statement 对象和 ResultSet 对象，都占用一定的 JDBC

当完成对数据库的访问之后，应及时关闭这些对象，以释放所占用的资源。这些对象都提供了 close() 方法，关闭对象的次序与创建对象的次序正相反

```
rs.close();
stmt.close();
conn.close();
```

JDBC 数据库操作

增加数据
修改数据
删除数据
查询数据
批处理

增加数据

使用 Statement 对象提供的带参数的 executeUpdate) () 方法

通过 PreparedStatement 对象提供的无参数的executeUpdate) () 方法

修改数据

同实现增加操作的方法基本相同

eg

使用 **Statement** 对象实现修改 users 表中用户名为zhangping 的用户，将其密码修改为 654321

```
Statement stmt = conn.createStatement ();
String sql = "update users set u_pwd ='654321' where u_name =' zhangping '";
int temp = stmt.executeUpdate(sql);
```

使用 **PreparedStatement** 对象实现修改 users 表中用户名为 zhangping1 的用户，将其密码修改为

```
String sql = "update users set u_pwd=? where u_name=?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "654321");
pstmt.setString(2, "zhangping1");
int temp =pstmt.executeUpdate();
```

删除数据

使用 **Statement** 对象实现删除 users 表中用户名为zhangping 的用户

```
Statement stmt = conn.createStatement();
String sql = "delete from users where u_name ='zhangping'";
int temp = stmt.executeUpdate(sql);
```

使用 **PreparedStatement** 对象实现删除 users 表中用户名为 zhangping1 的用户

```
String sql = " delete from users where u_name =?";
PreparedStatement t pstmt = conn.prepareStatement( ( sql; );
pstmt.setString (1, "zhangping1");
int temp = pstmt.executeUpdate
```

查询数据

-JDBC 同样提供了两种实现数据查询的方法

-使用 Statement 对象提供的带参数的 executeQuery)() 方法;

-通过 PreparedStatement 对象提供的无参数的executeQuery)() 方法。

-使用 SELECT 命令实现对数据的查询操作，查询的结果集使用 ResultSet 对象保存。

批处理

JDBC 使用 Statement 对象和 PreparedStatement 对象的相应方法实现批处理

（1）使用 addBatch(sql) 方法，将需要执行的 SQL 命令添加到批处理中。
（2）使用 executeBatch() 方法，执行批处理命令。
（3）使用 clearBatch()方法，清空批处理队列。

批量执行静态的 SQL
批量执行动态的 SQL
批量执行混合模式的 SQL

批量执行静态 SQL

使用 Statement 对象的 addBatch() 方法可以批量执行静态 SQL

优点是可以向数据库发送**多条**不同的 SQL 语句

缺点是 SQL 语句没有预编译，**执行效率较低**，并且当向数据库发送多条语句相同，但仅参数不同的 SQL 语句时，需重复使用多条相同的 SQL 语句

批量执行动态的 SQL

批量执行动态 SQL ，需要使用 PreparedStatement 对象的addBatch() 方法来实现批处理

优点是发送的是**预编译**后的 SQL 语句，**执行效率高。**

缺点是只能应用在 SQL 语句相同，但参数不同的批处理中

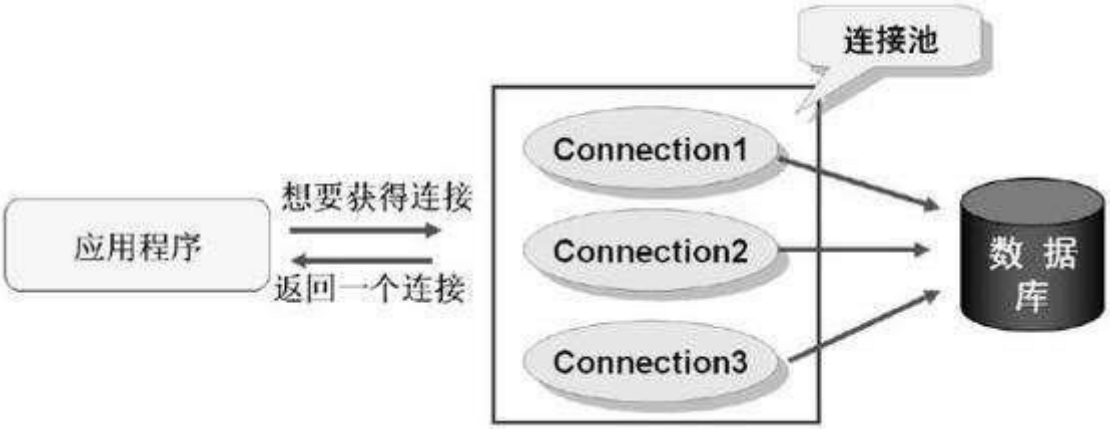
因此此种形式的批处理经常用于在同一个表中批量更新表中的数据

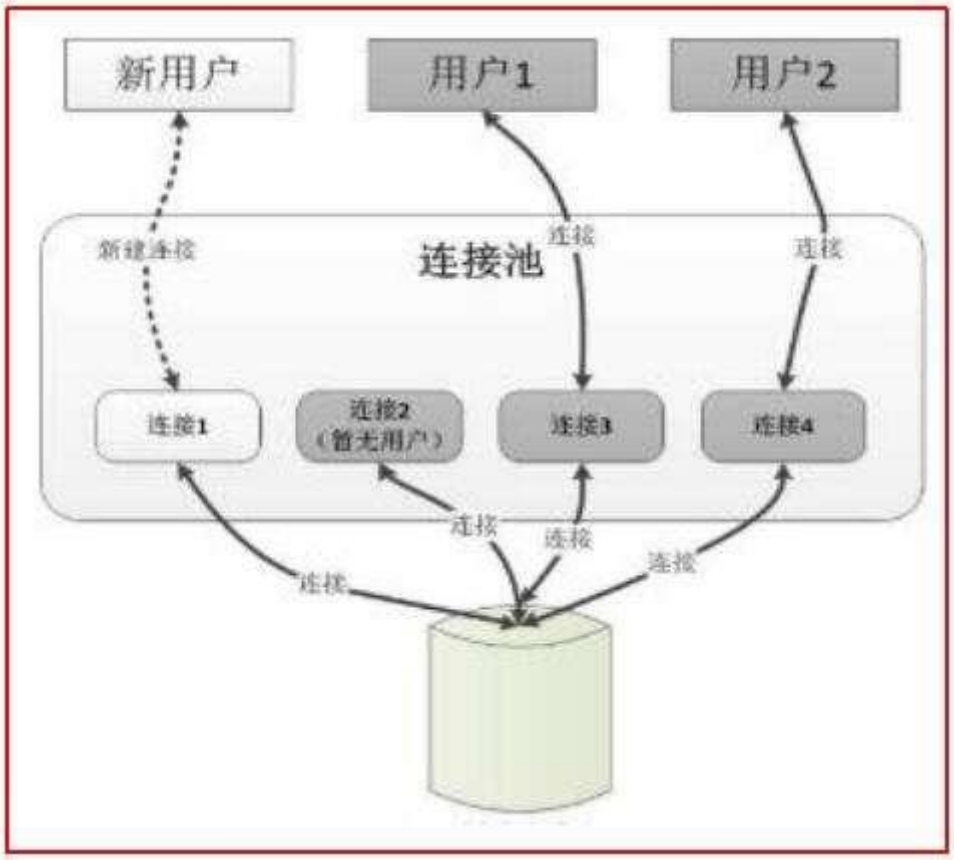
批量执行混合模式的 SQL

使用 PreparedStatement 对象的 addBatch) () 方法还可以实现混合模式的批处理，既可以执行批量执行动态 SQL，同时也可以批量执行静态 SQL

数据源

数据库连接池的基本思想就是为数据库连接建立一个“**缓冲池**”，预先在“缓冲池”中放入一定数量的连接。当需要建立数据库连接时，只需从“缓冲池”中取出一个，使用完毕之后再放回去。





数据源（Data Source）是目前 Web 开发中获取数据库连接的首选方法。这种方法是首先创建一个数据源对
象，由数据源对象事**先建立若干连接对象**，通过连接池管理这些连接对象。

JDBC示例

加载JDBC驱动，使用Java连接university数据库，查询instructor表中salary大于70000的教师信息，按照dept_name的升序排列后用System.out.println()输出，各列数据之间用|分隔。
ID|name|dept_name|salary将编写的Java类代码贴在下方输入框中。

```
import java.sql.*;
public class GaussDBMySQLDemo {
    // MySQL 8.0 以上版本 - JDBC 驱动名及数据库 URL
    static final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
    static final String DB_URL = "jdbc:mysql://localhost:3306/university?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";
    // 数据库的用户名与密码，需要根据自己的设置
    static final String USER = "root";
    static final String PASS = "123";
    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        try{
            // 注册 JDBC 驱动
            Class.forName(JDBC_DRIVER);
            // 打开链接
            System.out.println("连接数据库...");
```

```java
            conn = DriverManager.getConnection(DB_URL,USER,PASS);
            // 执行查询
            System.out.println(" 实例化Statement对象...");
            stmt = conn.createStatement();
            String sql;
            sql = "SELECT ID,`name`,dept_name,salary FROM instructor WHERE salary > 
70000 ORDER BY dept_name";
            ResultSet rs = stmt.executeQuery(sql);
            System.out.println("ID|name|dept_name|salary");
            // 展开结果集数据库
            while(rs.next()){
                // 通过字段检索
                int id  = rs.getInt("ID");
                String name = rs.getString("name");
                String dept_name = rs.getString("dept_name");
                int salary  = rs.getInt("salary");
                // 输出数据
                System.out.print(id + "|");
                System.out.print(name + "|");
                System.out.print(dept_name + "|");
                System.out.print(salary);
                System.out.print("\n");
            }
            // 完成后关闭
            rs.close();
            stmt.close();
            conn.close();
        }catch(SQLException se){
            // 处理 JDBC 错误
            se.printStackTrace();
        }catch(Exception e){
            // 处理 Class.forName 错误
            e.printStackTrace();
        }finally{
            // 关闭资源
            try{
                if(stmt!=null) stmt.close();
            }catch(SQLException se2){
            }// 什么都不做
            try{
               if(conn!=null) conn.close();
            }catch(SQLException se){
                se.printStackTrace();
            }
        }
        System.out.println("Goodbye!");
    }
}
```

（1）在Univerisity中编写一个带输入参数和输出参数的存储过程demoSp，功能自定，给出存储过程定义。
（2）编写Java代码调用存储过程，指定输入参数，获取输出参数后输出。给出调用存储过程的关键代码。

（1）

```
DROP PROCEDURE if EXISTS demoSp;
CREATE PROCEDURE demoSp(IN inputParam INT, OUT outputParam INT)
BEGIN
SELECT COUNT(*) INTO outputParam FROM instructor WHERE salary > inputParam;
END;
```

（2）

```
import java.sql.*;
import java.sql.CallableStatement;
public class GaussDBMySQLDemo {
    // MySQL 8.0 以上版本 - JDBC 驱动名及数据库 URL
    static final String JDBC_DRIVER = "com.mysql.cj.jdbc.Driver";
    static final String DB_URL = "jdbc:mysql://localhost:3306/university?
useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC";
    // 数据库的用户名与密码，需要根据自己的设置
    static final String USER = "root";
    static final String PASS = "123";
    public static void main(String[] args) {
        Connection conn = null;
        Statement stmt = null;
        try{
            // 注册 JDBC 驱动
            Class.forName(JDBC_DRIVER);
            // 打开链接
            System.out.println("连接数据库...");
            conn = DriverManager.getConnection(DB_URL,USER,PASS);
            // 执行查询
            System.out.println(" 实例化Statement对象...");
            CallableStatement cStmt = conn.prepareCall("{call demoSp(?, ?)}");
            cStmt.setInt(1,70000);
            cStmt.execute();
            int outputvalue = cStmt.getInt(2);
            System.out.println(outputvalue);
            cStmt.close();
            conn.close();
        }catch(SQLException se){
            // 处理 JDBC 错误
            se.printStackTrace();
        }catch(Exception e){
            // 处理 Class.forName 错误
            e.printStackTrace();
        }finally{
            // 关闭资源
            try{
                if(stmt!=null) stmt.close();
            }catch(SQLException se2){
            }// 什么都不做
            try{
                if(conn!=null) conn.close();
            }catch(SQLException se){
                se.printStackTrace();
            }
```

```
        }
        System.out.println("Goodbye!");
    }
}
```

索引

InnoDB 存储引擎支持的索引（使用 B+Tree 作为索引结构）
普通索引（ INDEX ）： 最基本的索引类型，值可以为空，没有唯一性限制
唯一值索引（ UNIQUE ）： 索引列的所有值都只能出现一次，即必须唯一，值可以为空
主键索引（ PRIMARY KEY ）： 为唯一性索引，必须指定为PRIMARY KEY ，每个表只能有一个主键（系统自动创建索引）
全文索引（ FULLTEXT ）： 可以在 varchar 、 char 、 text 类型的列上创建，以便能够根据快速地查询数据量较大的字符串类型的字段，目前不支持中文

```
CREATE [UNIQUE | FULLTEXT | SPATIAL] INDEX 索引名
    [ 索引类型 ]
    ON 表名 ( 索引列名 ...)
    [索引选项] ...
索引列名=:
列名 [(长度)] [ASC | DESC]
```

```
CREATE [TEMPORARY] TABLE [IF NOT EXISTS] 表名
    [ ( [ 列定义 ] , ... | [ 索引定义] ) ]
    [ 表选项 ] [select语句];
```

ALTER TABLE e tbl_name ADD PRIMARY KEY (column_list)
ALTER TABLE e tbl_name ADD UNIQUE index_name (column_list)
ALTER TABLE e tbl_name ADD INDEX index_name (column_list)
ALTER TABLE e tbl_name ADD FULLTEXT index_name (column_list)

（1）使用DROP INDEX删除

语法格式：

```
DROP INDEX 索引名 ON 表名
```

（2）使用ALTER TABLE删除

语法格式：

```
ALTER [IGNORE] TABLE 表名
......
 | DROP PRIMARY KEY
                                    /*删除主键*/
 | DROP {INDEX|KEY} 索引名
                          /*删除索引*/
 | DROP FOREIGN KEY fk_symbol
                          /*删除外键*/
```

示例

1)在test_user表上为username列建立普通索引，名字为idx_username。注意：　由于该表数据量较多，建立索引需要一些时间。

```
ALTER TABLE test_user ADD INDEX idx_username (username);
```

2)删除test_user表上的普通索引idx_username，写出对应的SQL语句。

```
    DROP INDEX idx_username ON test_user;
```

3)删除test_user表上的普通索引idx_username，写出对应的SQL语句

```
    DROP INDEX idx_username ON test_user;
```

4)在test_user表上建立唯一索引uk_username，写出SQL语句

```
    ALTER TABLE test_user ADD UNIQUE uk_username (username);
```

5)下面的SQL语句是否能够命中唯一索引uk_username，解释原因。SELECT * FROM test_user WHERE username like '%USER';

```
    EXPLAIN SELECT * FROM test_user WHERE username like '%USER';
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows
1	SIMPLE	test_user	(Null)	ALL	(Null)	(Null)	(Null)	(Null)	99848

不能命中，使用了模糊查询，无法使用唯一索引

6)

（a）在telephone上建立普通索引idx_telephone，写出SQL语句（b）SQL语句SELECT * FROM test_user where telephone=13900000000是否能够命中普通索引idx_telephone？如果不能，请解释原因并说明应该如何修改这条语句使其命中索引。

（a）

```
ALTER TABLE test_user ADD INDEX idx_telephone (telephone);
```

（b）

id	select_type	table	partitions	type	possible_keys	key	key_len	ref
1	SIMPLE	test_user	(Null)	ALL	idx_telephone	(Null)	(Null)	(Null)

不能命中

修改为

EXPLAIN SELECT * FROM test_user where telephone='13900000000'

id	select_type	table	partitions	type	possible_keys	key	key_len	ref
1	SIMPLE	test_user	(Null)	ref	idx_telephone	idx_telephone	47	const

避免类型转换以命中普通索引idx_telephone

事务管理

从 MySQL 4.1 开始支持事务，事务由作为一个单独单元的一
个或多个 SQL 语句组成

银行例子：

在银行转账过程中，如果要把 1000 元从账号“123” 转到账号 “456”，则需要先后执行以下
两条 SQL 命令：
update account set value=value+1000 where accountno =456;
update account set value=value- - 1000 where accountno

如果账号 “123” 和账号 “456” 的当前余额都为500，转账前两账户余额信息如图6 6- -1

```
mysql> use transaction_test;
Database changed
mysql> create table account(
    ->   accountno int primary key,
    ->   value int unsigned
    -> )engine=innodb;
Query OK, 0 rows affected (0.00 sec)

mysql> insert into account values(123,500);
Query OK, 1 row affected (0.00 sec)

mysql> insert into account values(456,500);
Query OK, 1 row affected (0.00 sec)

mysql> select * from account;
+-----------+-------+
| accountno | value |
+-----------+-------+
|       123 |   500 |
|       456 |   500 |
+-----------+-------+
2 rows in set (0.00 sec)
```

当第一条 SQL 语句执行完后，账号 “456” 的当前余额为1500 。当第二条 SQL 语句执行时，由于当前余额不足 1000 ，所以 SQL 语句执行失败，当前账户余额应为 500 。这样就产生了数据不一致问题，转账后两账户余额信息如图6 6- -2 2 所示。

```
mysql> update account set value=value+1000 where accountno=456;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> update account set value=value-1000 where accountno=123;
ERROR 1264 (22003): Out of range value adjusted for column 'value' at row 1
mysql> select * from account;
+-----------+-------+
| accountno | value |
+-----------+-------+
|       123 |   500 |
|       456 |  1500 |
+-----------+-------+
2 rows in set (0.00 sec)
```

如果将上面两条 update 语句绑定到一起形成一个事务，那么这两条 update 语句或者都执行，或者都不执行，从而避免数据不一致问题。

事务的提交和回滚

在 MySQL 中，当一个会话开始时，系统变量AUTOCOMMIT 值为 1 ，即自动提交功能是打开的。

当任意一条 SQL 语句发送到服务器时， MySQL 服务器会立即解析、执行并将更新结果提交到数据库文件中 。

因此在执行事务时要首先关闭 MySQL 的自动提交，使用命令t “set autocommit =0;” 可以关闭 MySQL 的自动提交。这样只有事务中的所有操作都成功执行后，才提交所有操作，否则回滚所有操作。

当 MySQL 关闭自动提交后，可以使用 COMMIT 命令来完成事务的提交 。
COMMIT 语句使得从事务开始以来所执行的所有数据修改成为数据库的永久组成部分，也标志着一个事务的结束 。
使用命令t “start transaction;” 可以开启一个事务，该命令开启事务的同时会隐式地关闭 MySQL 自动提交

在 MySQL 中，事务是不允许嵌套的。如果在第一个事务里使用 start transaction 命令后，当开始第二个事务会自动提交第一个事务。 下面的语句在运行时都会隐式地执行一个 commit 命令 。
set autocommit =1 、 rename table 、e truncate table；
数据定义语句： create 、 alter 、 drop；
权限管理和账户管理语句： grant 、 revoke 、set password 、create user 、drop user 、rename user;
锁语句：lock tables 、unlock tables

事务的四大特性和隔离级别

数据库中的事务具有 ACID 属性，即原子性 （ Atomicity ） 、一致性 （ Consistency ） 、隔离性 （ Isolation ）和持永性 （ Durability ） 。
1 ．原子性
原子性意味着每个事务都必须被认为是一个不可分割的单元，事务中的操作必须同时成功事务才是成功的。如果事务中的任何一个操作失败，则前面执行的操作都将回滚，以保证数据的整体性没有受到影响 。

【例6- -3】事务的原子性。
１、首先开启一个事务，在 account 账户信息表中插入二条账户信息 （ 555 ， 500 ）和 （ 666 ， 500 ），然后提交事务 。
２、再开启第二个事务，在 account 账户信息表中插入二条账户信息 （ 777 ， 500 ）和 （ 888 ， - - 500 ）， 然后回滚事务 。

2．一致性
事务的一致性保证了事务完成后，数据库能够处于一致性状态。如果事务执行过程中出现错误，那么数据库中的所有变化将自动地回滚，回滚到另一种一致性状态。
在 MySQL 中一致性主要由 MySQL 的日志机制处理，它记录了数据库的所有变化，为事务恢复提供了跟踪记录。
如果系统在事务处理中发生错误， MySQL 恢复过程将使用这些日志来发现事务是否已经完全成功地执行，是否需要返回。
一致性保证了数据库从不返回一个未处理完的事务。

3．隔离性
事务的隔离性确保多个事务并发访问数据时，各个事务不能相互干扰。
系统中的每个事务在自己的空间执行，并且事务的执行结果只有在事务执行完才能看到。
即使系统中同时执行多个事务，事务在完全执行完之前，其他事务是看不到结果的。
在多数事务系统中，可以使用页级锁定或行级锁定来隔离不同事务的执行。

事务的隔离性不允许多个事务同时修改相同的数据，所以第一个事务执行完 update 命令但未提交时第二个事务的 update 命令需要等待。当第一个事务提交后，第二个事务才会执行 update 命令。

4 .持久性
事务的持久性意味着事务一旦提交，其改变会永久生效，不能再被撤销。
即使系统崩溃，一个提交的事务仍然存在。
MySQL 通过保存所有行为的日志来保证数据的持久性，数据库日志记录了所有对于表的更新操作。

事务的隔离级别

事务的隔离级别是事务并发控制的整体解决方案，是综合利用各种类型的锁机制解决并发问题 。
每个事务都有一个隔离级，它定义了事务彼此之间隔离和交互的程度 。
在 MySQL 中提供了4 种隔离级别： read uncommitted （读取未提交的数据 ）、 read committed （读取提交的数据 ）、repeatable read （可重复读）和 serializable （串行化 ）。
其中，read uncommitted 的隔离级别最低， serializable 的隔离级别最高 ， 4 种隔离级别逐渐增加。

1． read uncommitted （读取未提交的数据 ）提供了事务之间的最小隔离程度，处于这个隔离级别的事务可以读到其他事务还没有提交的数据 。
2． read committed （读取提交的数据 ）处于这一级别的事务可以看见已经提交事务所做的改变，这一隔离级别要低于repeatable read （可重复读 ）。

3． repeatable read （可重复读 ）
这是 MySQL 默认的事务隔离级别 ， 它确保在同一事务内相同的查询语句其执行结果总是相同的 。
4． serializable （串行化）
这是最高级别的隔离，它强制事务排序，使事务一个接一个地顺序执行 。
查看当前 MySQL 会话的事务隔离级别可以使用命令 “select@@ session.tx_isolation ;” 。 查看 MySQL 服务实例全局事务隔离级别可以使用命令t “select @@ global.tx_isolation ;”

脏读

一个事务可以读到另一个事务未提交的数据则为脏读。如果将事务的隔离级别设置为 read uncommitted ，则可能出现脏读、不可重复读和幻读等问题 。
将事务的隔离级别设置为read committed 则可以避免脏读，但可能出现不可重复读以及幻读等问题 。

不可重复读

在同一个事务中，两条相同的查询语句其查询结果不一致 。
当一个事务访问数据时，另一个事务对该数据进行修改并提交，导致第一个事务两次读到的数据不一样 。
当事务的隔离级别设置为d read committed 时可以避免脏读，但可能会出现不可重复读 。
将事务的隔离级别设置为e repeatable read ， 则可以避免脏读和不可重复读 。

幻读

幻读是指当前事务读不到其他事务已经提交的修改。
将事务的隔离级别设置为 repeatable read 可以避免脏读和不可重复读，但可能会出现幻读。
将事务的隔离级别设置为 serializable ， 可以避免幻读。

code

题目：请创建一个新表instructor(ID varchar(10), name varchar(20), dept_name varchar(10), salary int)，设置ID为主键，并插入一条教师信息（'12121', 'Wu', 'Comp. Sci.', 5000），进行MySQL隔离级别验证实验。请按照如下步骤完成实验并回答相应问题，在完成实验之后请删除该instructor表。

1）将instructor表中编号12121的名为Wu的教工的salary设置为10000元

2）同时开两个命令行界面，称为A和B，在A、B两个窗口都执行下面的语句，设定隔离级别为“读未提交”。set transaction_isolation='read-uncommitted';可以执行select @@session.transaction_isolation;验证设定是否已经成功。

3）在A窗口执行如下语句：begin;select * from instructor;可以看到所有教工的信息，编号12121的名为Wu的教工的salary当前值应该为10000

4）在B窗口执行如下语句：begin;update instructor set salary = salary - 500 where id= 12121;select * from instructor;

5）在A窗口执行如下语句：select * from instructor;

1.请回答如下问题：

Q1：是否可以看到编号12121的名为Wu的教工的salary值被B窗口修改但未提交的结果，回答是/否。

Q2：这种现象的名称是？

6）在B窗口执行如下语句：rollback;

7）在A窗口执行如下语句：update instructor set salary = salary - 500 where id= 12121;select * from instructor;

2. 请回答如下问题：Q：当前编号12121的名为Wu的教工的salary值为多少？

8）在A窗口执行如下语句：rollback;select * from instructor;

9）在A、B两个窗口都执行下面的语句，设定隔离级别为“读已提交”。set transaction_isolation='read-committed';可以执行select @@session.transaction_isolation;验证设定是否已经成功。

10）在A窗口执行下面语句：begin;select * from instructor;

11）在B窗口执行下面语句：begin;select * from instructor;update instructor set salary = salary - 500 where id= 12121;

12）在A窗口执行如下语句：select * from instructor;

3.请回答如下问题：

Q1：当前编号12121的名为Wu的教工的salary值为多少？

Q2：是否可以看到编号12121的名为Wu的教工的salary值被B窗口修改但未提交的结果，回答是/否

13）在B窗口执行下面语句：commit;select * from instructor;

14）在A窗口执行如下语句：select * from instructor;commit;

4.请回答如下问题：

Q1：当前编号12121的名为Wu的教工的salary值为多少？

Q2：是否可以看到编号12121的名为Wu的教工的salary值被B窗口修改并已提交的结果，回答是/否

Q3：A窗口当前两次的select * from instructor;查询结果是否一致？

Q4：这种现象的名称是？

15）在A、B两个窗口都执行下面的语句，设定隔离级别为“可重复读”。set transaction_isolation='repeatable-read';可以执行select @@session.transaction_isolation;验证设定是否已经成功。

16）在A窗口执行下面语句：begin;select * from instructor;

16）在B窗口执行下面语句：begin;select * from instructor;update instructor set salary = salary - 500 where id= 12121;commit;select * from instructor;

17）在A窗口执行如下语句：select * from instructor;

5.请回答如下问题：

Q1：当前编号12121的名为Wu的教工的salary值为多少？

Q2：是否可以看到编号12121的名为Wu的教工的salary值被B窗口修改并已提交的结果，回答是/否

Q3：A窗口当前两次的select * from instructor;查询结果是否一致？

18）在A窗口执行如下语句：update instructor set salary = salary - 500 where id= 12121;commit;select * from instructor;

6.请回答如下问题：

Q1：当前编号12121的名为Wu的教工的salary值为多少？

19）在A窗口执行如下语句：begin;select * from instructor;

20）在B窗口执行下面语句：begin;insert into instructor values(99999,'test','Comp. Sci.',10000);commit;select * from instructor;

21）在A窗口执行如下语句：select * from instructor;

7. 请回答如下问题：

Q1：是否可以看到B窗口新插入并已提交的结果，回答是/否

22）在A窗口执行如下语句：insert into instructor values(99999,'test','Comp. Sci.',10000);

8.请回答如下问题：

Q1：A窗口是否能插入此条数据，回答是/否

Q2：这种现象的名称是？

23）在A窗口执行如下语句：update instructor set salary = 6666 where id=99999;select * from instructor;commit;

9. 请回答如下问题：

Q1：A窗口是否更新了一条自己会话中查不到的数据，回答是/否

```
1. 请回答如下问题：

Q1：是否可以看到编号12121的名为Wu的教工的salary值被B窗口修改但未提交的结果，回答是/否。

A1：是
```

Q2：这种现象的名称是？

A2：脏读

2. 请回答如下问题：

Q：当前编号12121的名为Wu的教工的salary值为多少？

A：9500

3. 请回答如下问题：

Q1：当前编号12121的名为Wu的教工的salary值为多少？

A1：10000

Q2：是否可以看到编号12121的名为Wu的教工的salary值被B窗口修改但未提交的结果，回答是/否

A2：否

4. 请回答如下问题：

Q1：当前编号12121的名为Wu的教工的salary值为多少？

A1：9500

Q2：是否可以看到编号12121的名为Wu的教工的salary值被B窗口修改并已提交的结果，回答是/否

A2：是

Q3：A窗口当前两次的select * from instructor;查询结果是否一致？

A3：否

Q4：这种现象的名称是？

A4：不可重复读

5. 请回答如下问题：

Q1：当前编号12121的名为Wu的教工的salary值为多少？

A1：9500

Q2：是否可以看到编号12121的名为Wu的教工的salary值被B窗口修改并已提交的结果，回答是/否

A2：否

Q3：A窗口当前两次的select * from instructor;查询结果是否一致？

A3：是

6. 请回答如下问题：

Q1：当前编号12121的名为Wu的教工的salary值为多少？

A1：8500

7. 请回答如下问题：

Q1：是否可以看到B窗口新插入并已提交的结果，回答是/否

A1：否

8. 请回答如下问题：

Q1：A窗口是否能插入此条数据，回答是/否

A1：否

Q2：这种现象的名称是？

A2：幻读

9. 请回答如下问题：

Q1：A窗口是否更新了一条自己会话中查不到的数据，回答是/否

A1：是